



Seminario Curso:

Interactive Web-Based Data Visualization: Plotly with R

P.L. 2

PL2 Interactive Graphics.

1 Introducción.

2 Creación de vistas.

3 Publicación de vistas.

4 Combinando múltiples vistas.

5 Conectando múltiples vistas.

1.- Introducción

- La fase exploratoria de un flujo de trabajo de ciencia de datos requiere mucha iteración entre la manipulación de datos, la visualización y el modelado.
- Los paquetes R como el tidyverse permiten limitar la carga cognitiva sin eliminar los beneficios de realizar el análisis a través del código (dplyr, ggplot2, plotly, ...).

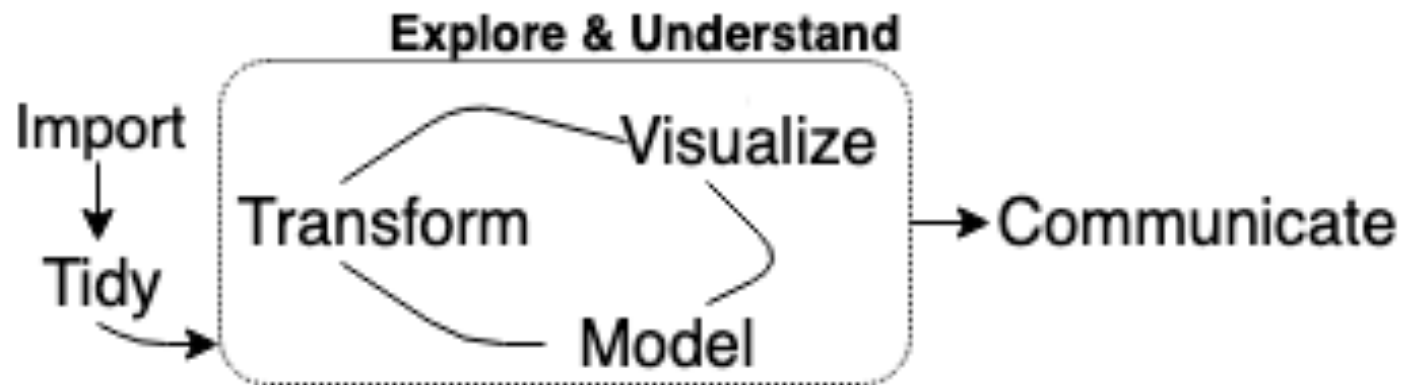


Diagram from: *R for Data Science* (Grolemund & Wickham 2016)

1.- Introducción

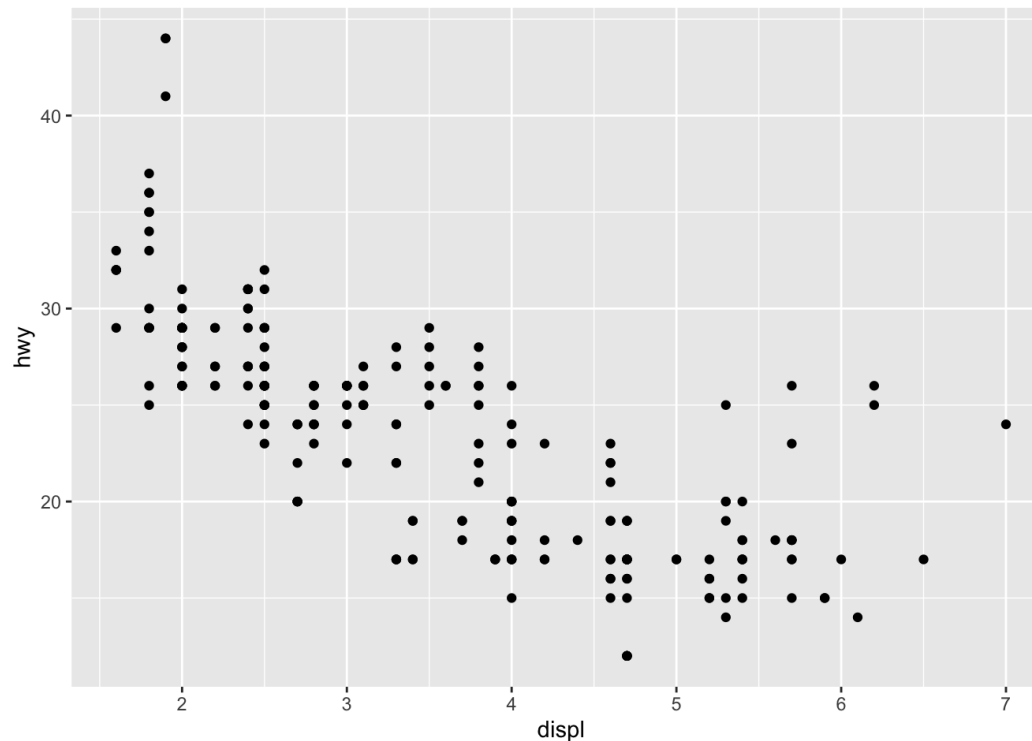
- En muchos casos, gráficos interactivos en la web permiten presentar los hallazgos más importantes.
- Los gráficos interactivos han mostrado ser una gran promesa en la ayuda en la exploración de datos de grandes dimensiones.
- Tareas que requieren interactividad:
 - Identificar la estructura que de otra forma estaría oculta.
 - Diagnosticar modelos y comprender algoritmos.
 - Ayudar en la búsqueda de información rápidamente sin que las preguntas estén completamente especificadas.

1.- Introducción

- Interfaz Gráfica de Usuario (GUI) para crear gráficos interactivos: DataDesk, GGobi, Mondrian, JMP, Tableau.
- PROBLEMA: Los sistemas basados en GUI no encajan en un flujo de trabajo basado en códigos:
 - Cualquier tarea que se complete a través de una GUI probablemente no pueda ser reproducida sin la intervención humana.
 - Si en algún momento los datos cambian posiblemente sea necesario recordar con precisión cómo reproducir el resultado.
 - Además, los sistemas basados en GUI son típicamente sistemas "cerrados" que no permiten ser fácilmente personalizados, extendidos o compartidos.

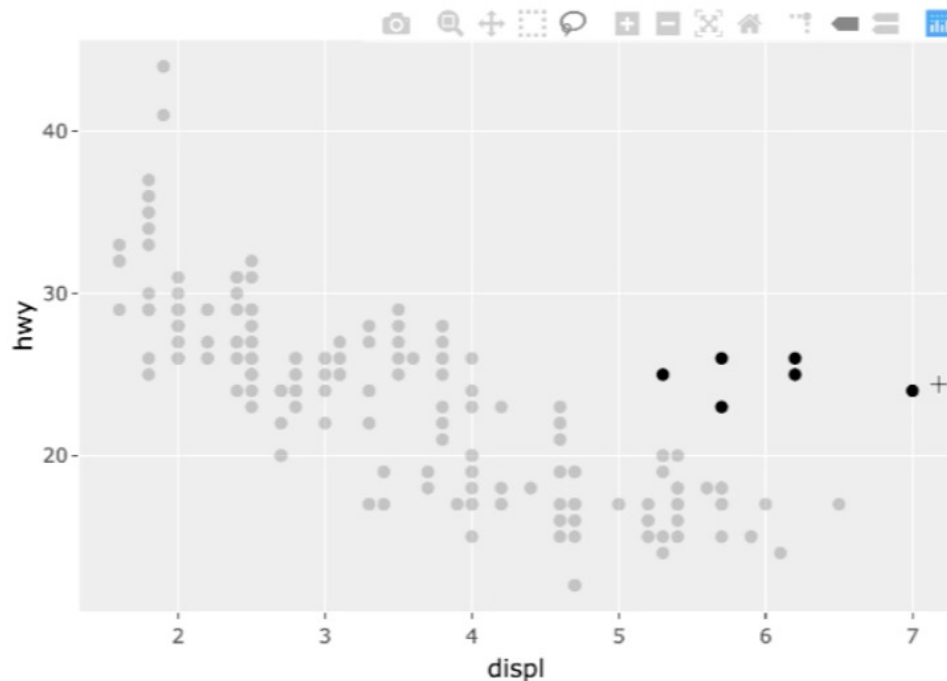
1.- Introducción

- La **programación de gráficos interactivos**: flexible la consulta, flexible la exploración.
- Un ejemplo:
 - `library(ggplot2)`
 - `ggplot(mpg, aes(displ, hwy)) + geom_point()`



1.- Introducción

- Añadimos interactividad:
 - `library(plotly)`
 - `m <- highlight_key(mpg)`
 - `p <- ggplot(m, aes(displ, hwy)) + geom_point()`
 - `gg <- highlight(ggplotly(p), "plotly_selected")`
 - `crosstalk::bscols(gg, DT::datatable(m))`



Show 10 entries

	manufacturer	model	displ	year	cyl	trans
24	chevrolet	corvette	5.7	1999	8	manual(m6)
25	chevrolet	corvette	5.7	1999	8	auto(l4)
26	chevrolet	corvette	6.2	2008	8	manual(m6)
27	chevrolet	corvette	6.2	2008	8	auto(s6)
28	chevrolet	corvette	7	2008	8	manual(m6)
159	pontiac	grand prix	5.3	2008	8	auto(s4)

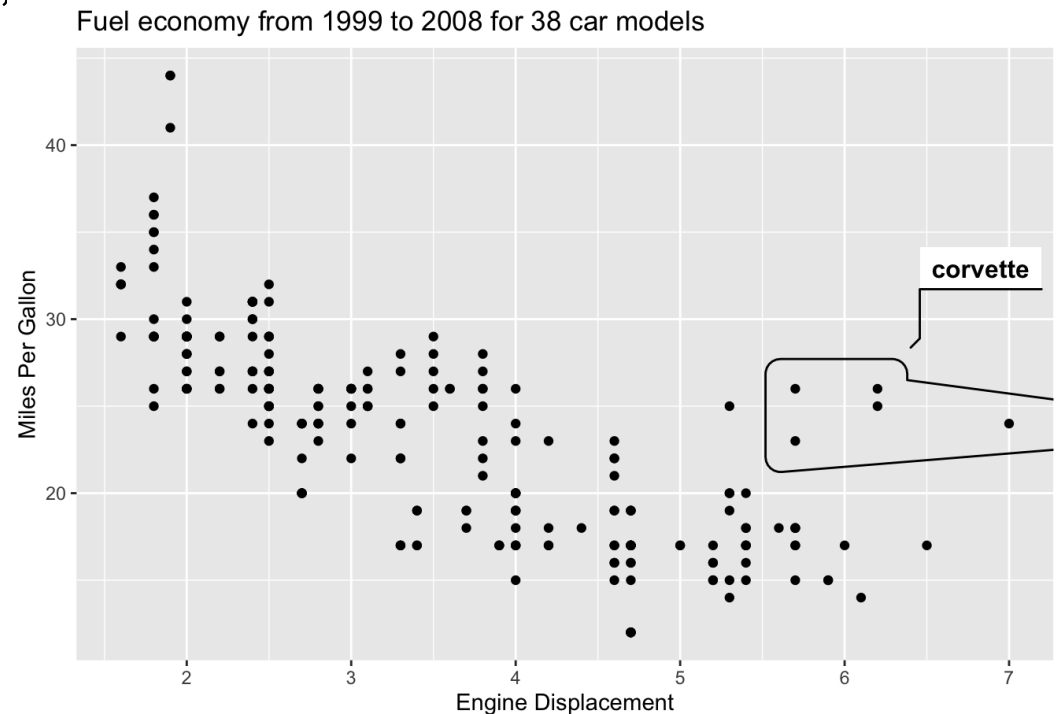
Showing 1 to 6 of 6 entries (filtered from 234 total entries)

1.- Introducción

- El gráfico basado en la web puede compartirse fácilmente con los colaboradores a través del correo electrónico y/o incorporarse dentro de un informe automatizado más grande o un sitio web.
- Funcionan perfectamente dentro de documentos más grandes de markdown, dentro de aplicaciones shiny, RStudio, cuadernos de Jupyter, R,
- Cuando es necesario mostrar los hallazgos es posible convertirlo en una gráfica estática.

1.- Introducción

- EJEMPLO:
 - library(ggforce)
 - ggplot(mpg, aes(displ, hwy)) +
 - geom_point() +
 - geom_mark_hull(aes(filter = model == "corvette", label = model)) +
 - labs(
 - title = "Fuel economy from 1999 to 2008 for 38 car models",
 - caption = "Source: <https://fuelconomy.gov/>",
 - x = "Engine Displacement",
 - y = "Miles Per Gallon"
 -)



1.- Introducción

- La programación de los gráficos interactivos puede no ser tan intuitiva como el uso de un sistema basado en una interfaz gráfica de usuario, **PERO** el esfuerzo puede merecer la pena en términos de automatización, escalado, “pervasividad” y flexibilidad.



Plotly

1.- Introducción

- This course (book) is not an island; there is no single resource that will allow you to master plotly (R). As you start to apply the techniques described in this class (book) to your own data you will soon find questions that I do not answer. This course (section) describes a few tips on how to get help, and to help you keep learning.

Wickham and Grolemund (2018)

PL2 Interactive Graphics.

1 Introducción.

2 Creación de vistas.

3 Publicación de vistas.

4 Combinando múltiples vistas.

5 Conectando múltiples vistas.

2.- Creación de vistas

INTRODUCCIÓN A PLOTLY:

- Hay dos formas de construir un gráfico con Plotly:

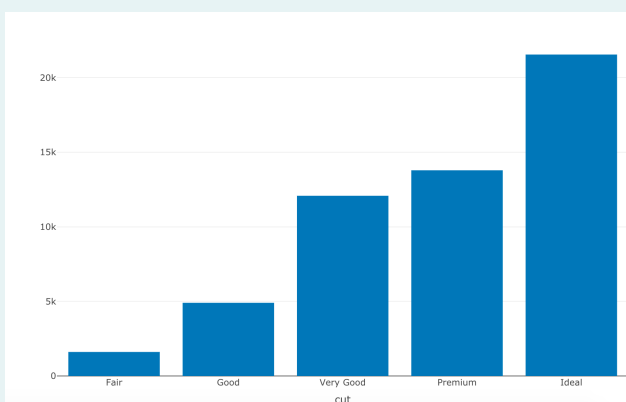
- Directamente ejecutar una instrucción plotly: *plot_ly()*, *plot_geo()*, *plot_mapbox()*, ...
- Crear un gráfico con ggplot y después utilizar la instrucción *ggplotly()*.

2.- Creación de vistas

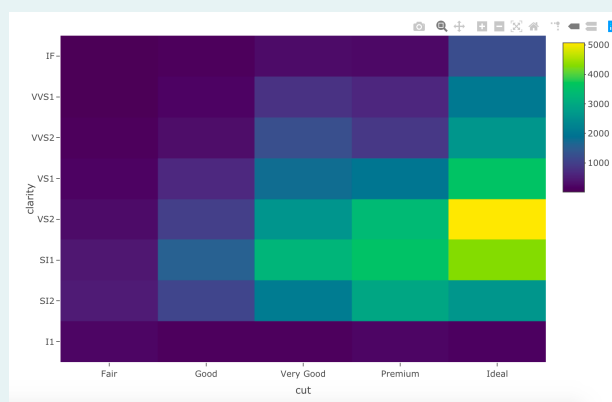
EJECUTANDO INSTRUCCIONES PLOTLY:

- Inspirado en la implementación de **ggplot2** y la Gramática de los Gráficos.
- EJEMPLOS: (usando el “dataset” *diamonds*):

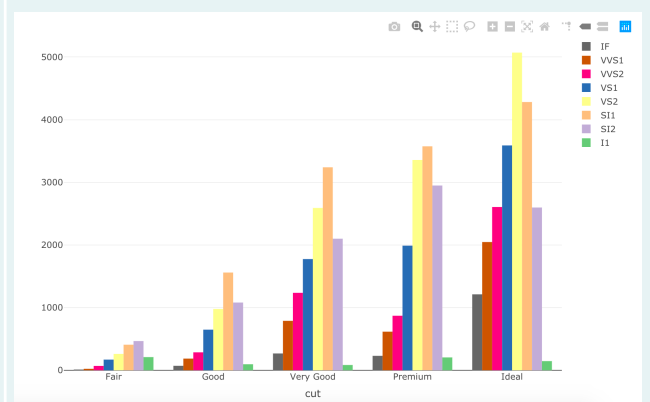
`plot_ly(diamonds,
x=~cut)`



`plot_ly(diamonds,
x=~cut, y=~clarity)`

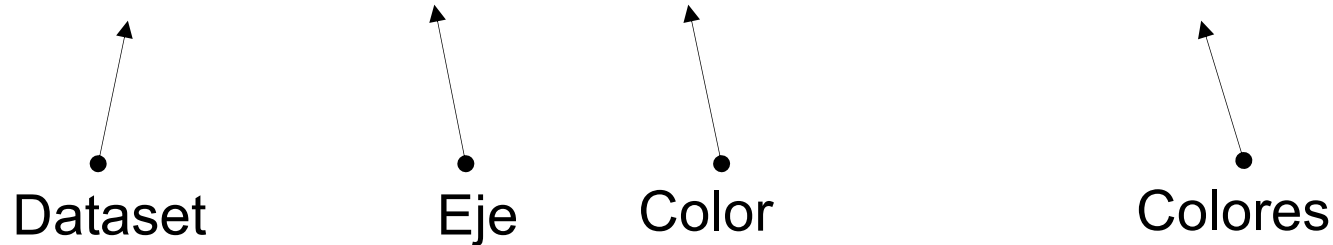


`plot_ly(diamonds,
x=~cut, color=~clarity,
colors="Accent")`



2.- Creación de vistas

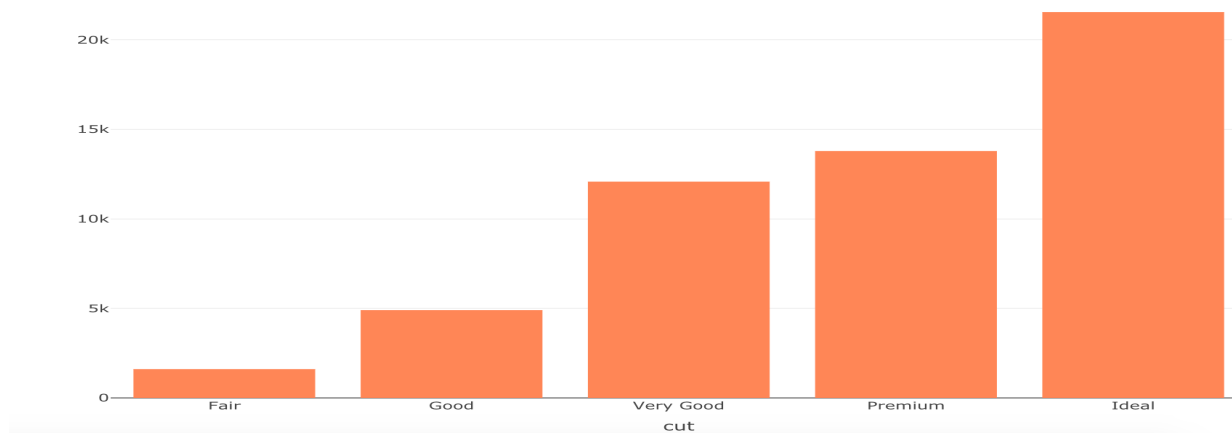
```
plot_ly(diamonds, x=~cut, color=~clarity, colors="Accent")
```



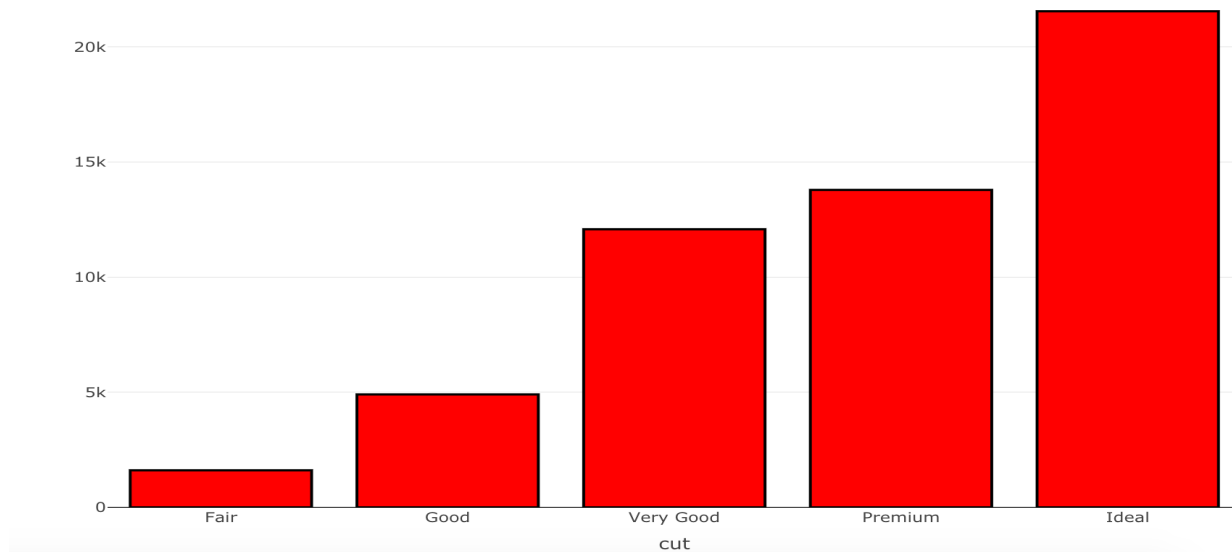
- **color**: permite asignar estéticas de color a un dato. Si se desea fijar un color utilizar la función `I()`.
- **colors**: permite fijar la gama de colores a utilizar en el mapeo de los datos asignados a **color** (colorbrewer2.org o `colorRamp()`).

2.- Creación de vistas

```
plot_ly(diamonds, x=~cut, color = "black")
```



```
plot_ly(diamonds, x=~cut, color = l("red"), stroke=l("black"),  
span=l(2))
```



2.- Creación de vistas

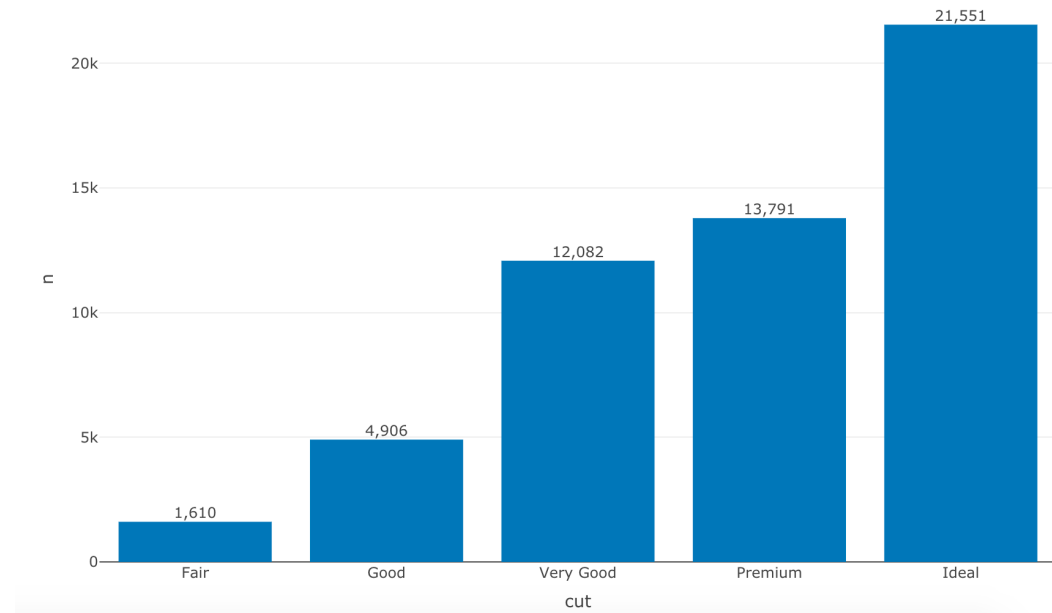
- **Plotly:** aproximación funcional a la gramática de gráficos por capas.
- EJEMPLO:
 - `layout(plot_ly(diamonds, x = ~cut), title = "My beatiful histogram")`
- PROBLEMA: lectura anidada (de dentro a afuera).
- SOLUCIÓN: utilizar el operador `%>%` para conseguir una lectura secuencial:
 - `diamonds %>% plot_ly(x = ~cut) %>% layout(title = "My beatiful histogram")`

2.- Creación de vistas

- La familia de funciones *add_**() permite añadir capas que mapean datos a objetos geométricos. Ejemplos:
 - *add_histogram()*, *add_bars()*, *add_markers()*, *add_lines()*, *add_histogram2D()*, *add_contour()*, *add_boxplot()*...
- Si no se especifica *plot_ly()* añade una capa en base a los parámetros de entrada que tiene instanciados.
- Cosas a tener en cuenta:
 - Los parámetros de *plot_ly()* los heredan los *add_**() (excepto si *inherit=FALSE*).
 - Las funciones de manipulación de datos de **dplyr** se pueden usar para transformar los datos de cualquier objeto **plotly**.

2.- Creación de vistas

- EJEMPLO:
 - `library(dplyr)`
 - `library(plotly)`
 - `diamonds %>%`
 - `plot_ly(x = ~cut) %>%`
 - `add_histogram() %>%`
 - `group_by(cut) %>%`
 - `summarise(n = n()) %>%`
 - `add_text(`
 - `text = ~scales::comma(n), y = ~n,`
 - `textposition = "top middle")`



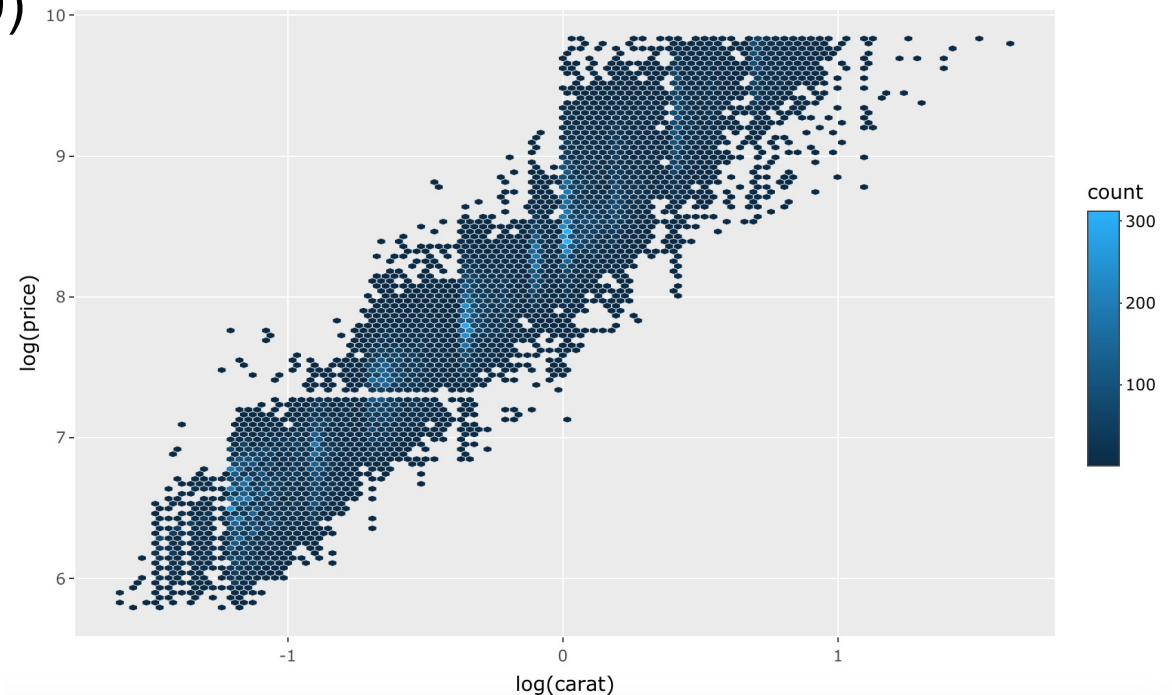
2.- Creación de vistas

INTRODUCCIÓN A PLOTLY:

- Hay dos formas de construir un gráfico con Plotly:
 - Directamente ejecutar una instrucción plotly: *plot_ly()*, *plot_geo()*, *plot_mapbox()*, ...
 - Crear un gráfico con ggplot y después utilizar la instrucción *ggplotly()*.

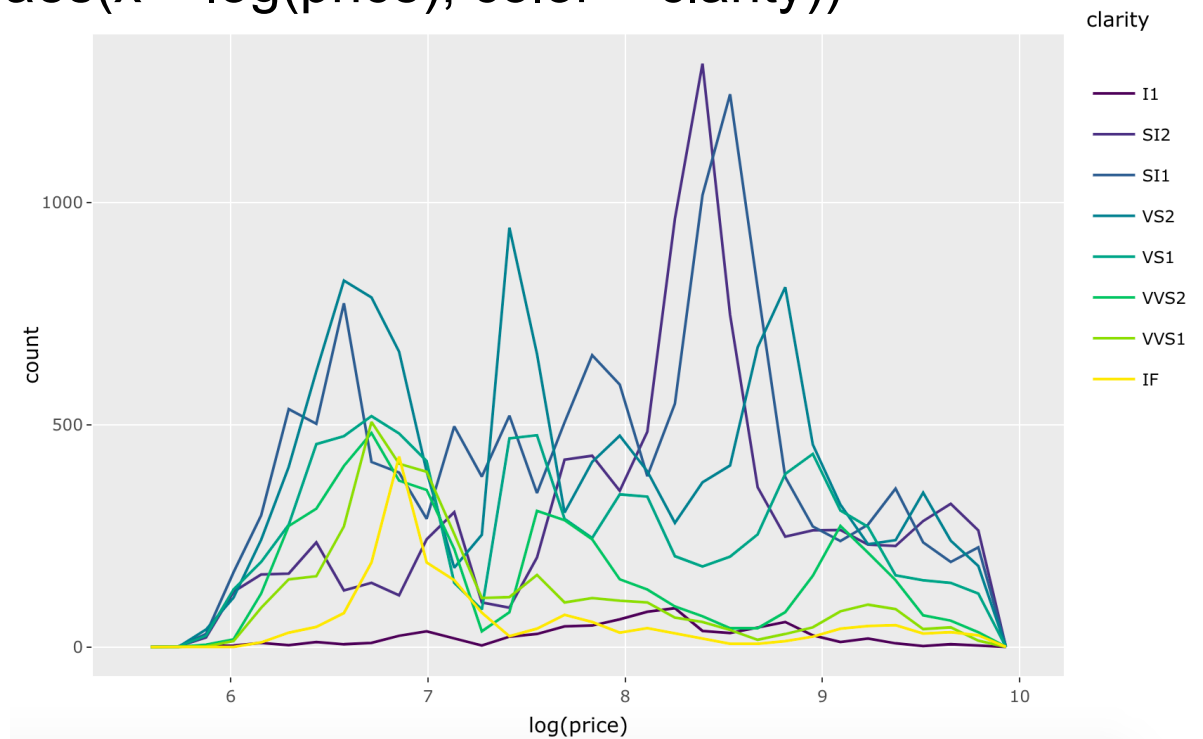
2.- Creación de vistas

- ***ggplotly()***: permite traducir cualquier gráfico *ggplot2* a *plotly* (añadiendo interactividad).
- EJEMPLO:
 - `p <- ggplot(diamonds, aes(x = log(carat), y = log(price))) +`
 - `geom_hex(bins = 100)`
 - `ggplotly(p)`



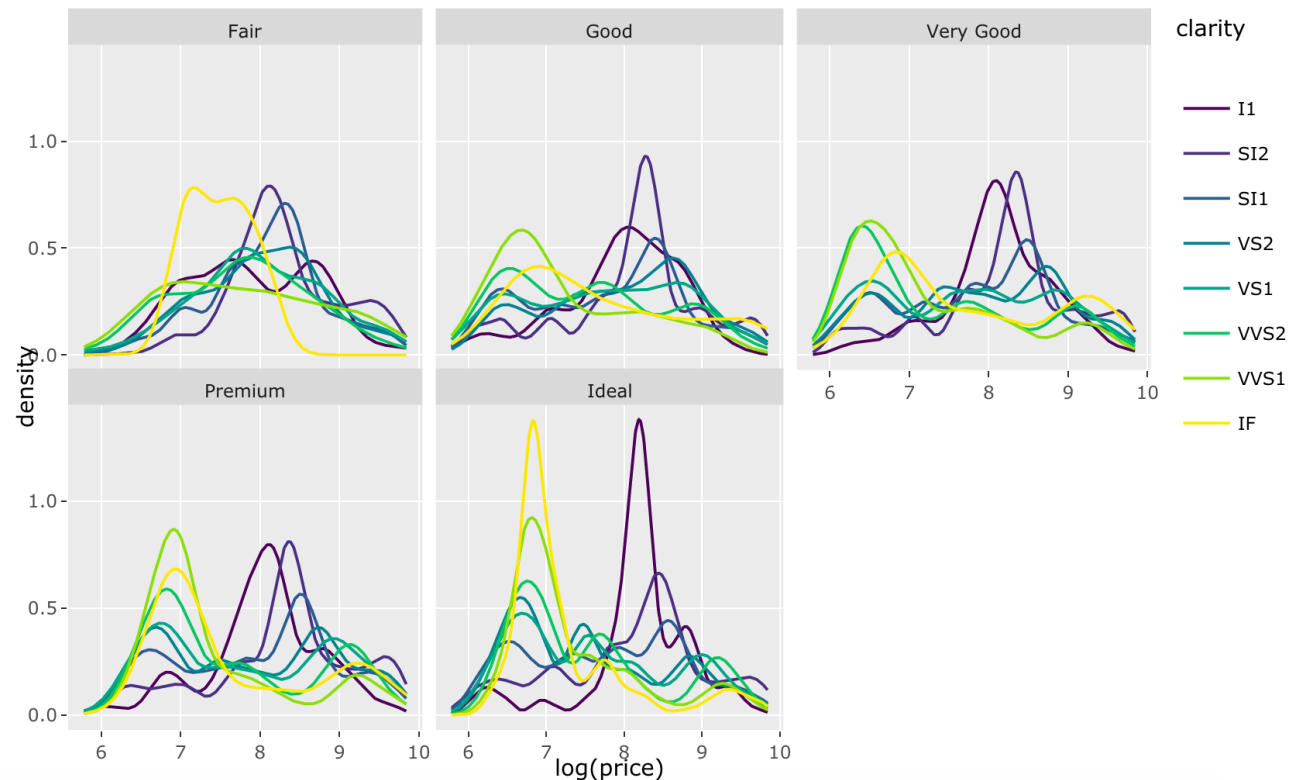
2.- Creación de vistas

- **ggplot2:** Tiene la habilidad de generar cualquier resumen estadístico para cualquier número de grupos en casi cualquier geometría.
- EJEMPLO: (distribución de diamantes por precio)
 - `p <- ggplot(diamonds, aes(x = log(price), color = clarity)) +`
 - `geom_freqpoly()`
 - `ggplotly(p)`



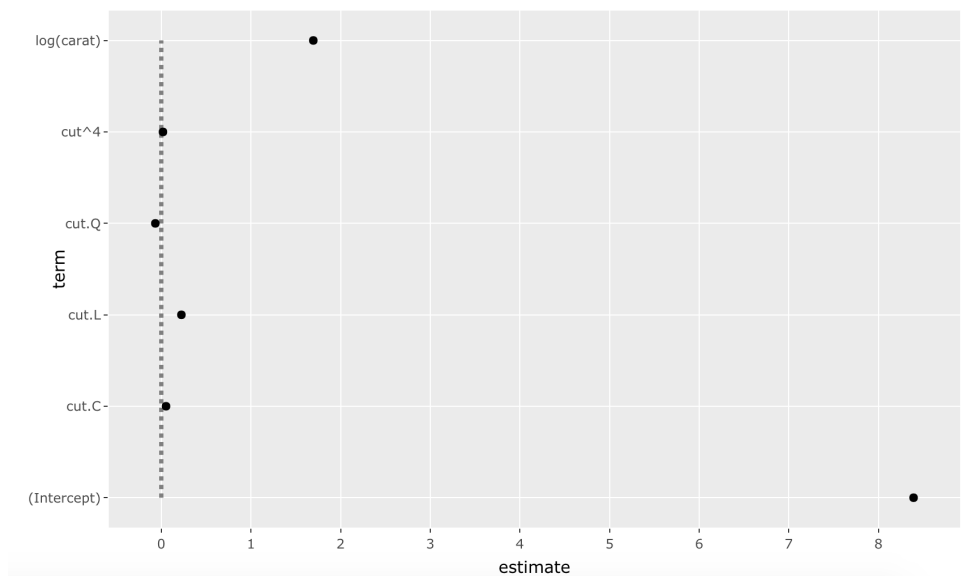
2.- Creación de vistas

- EJEMPLO (distribución de precios por corte):
 - `p <- ggplot(diamonds, aes(x = log(price), color = clarity)) +`
 - `geom_freqpoly(stat = "density") +`
 - `facet_wrap(~cut)`
 - `ggplotly(p)`



2.- Creación de vistas

- EJEMPLO (XAI -> qué influye más en la predicción del precio):
 - `library(GGally)`
 - `m <- lm(log(price) ~ log(carat) + cut, data = diamonds)`
 - `gg <- ggcoef(m)`
 - `# dynamicTicks means generate new axis ticks on zoom`
 - `ggplotly(gg, dynamicTicks = TRUE)`



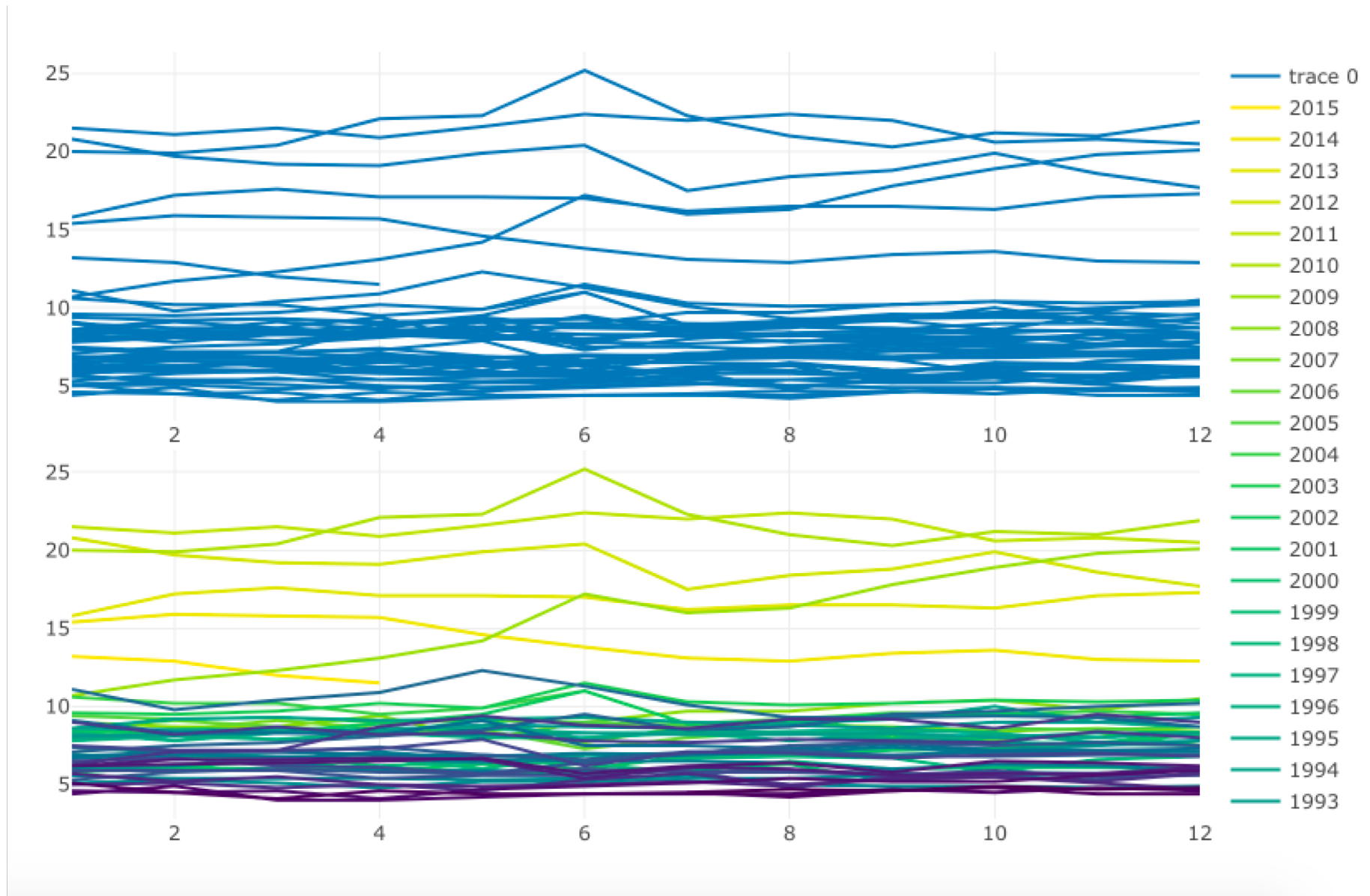
2.- Creación de vistas: “scattered plots”

- Los gráficos dispersos o **statter plots** representan información con geometrías básicas (punto, línea, texto, ...).
- Tipos:
 - *add_markers()*
 - *add_lines()*
 - *add_paths()*
 - *add_segments()*
 - *add_ribbons()*
 - *add_area()*
 - *add_polygons()*

2.- Creación de vistas: “scattered plots”

- Se suelen combinar con instrucciones **dplyr**:
 - *mutate()* para añadir columnas.
 - *group_by()* para crear grupos para visualización.
 - *summarise()* permite crear un “data frame” resumen.
- EJEMPLO:
 - *library(lubridate)*
 - *econ <- economics %>%*
 - *mutate(yr = year(date), mnth = month(date))*
 - *econ %>% group_by(yr) %>%*
 - *plot_ly(x = ~mnth, y = ~uempmed) %>% add_lines(text = ~yr)*
 - *plot_ly(econ, x = ~mnth, y = ~uempmed) %>%*
 - *add_lines(color = ~ordered(yr))*

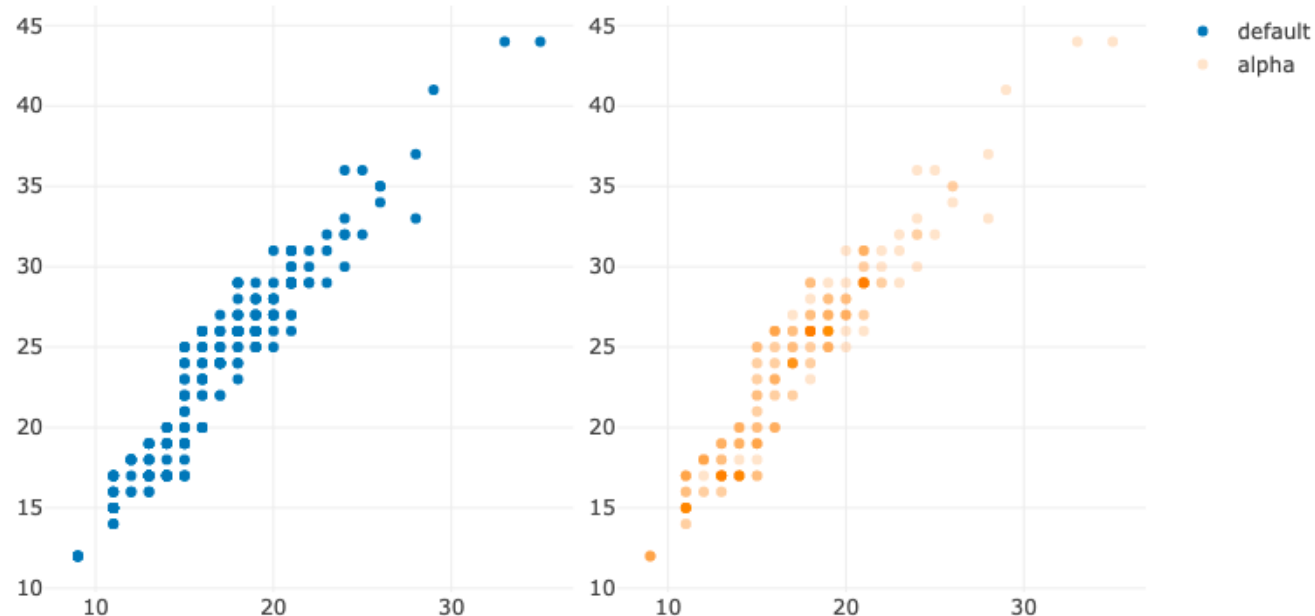
2.- Creación de vistas: “scattered plots”



2.- Creación de vistas: “scatter plots”

`add_markers()`:

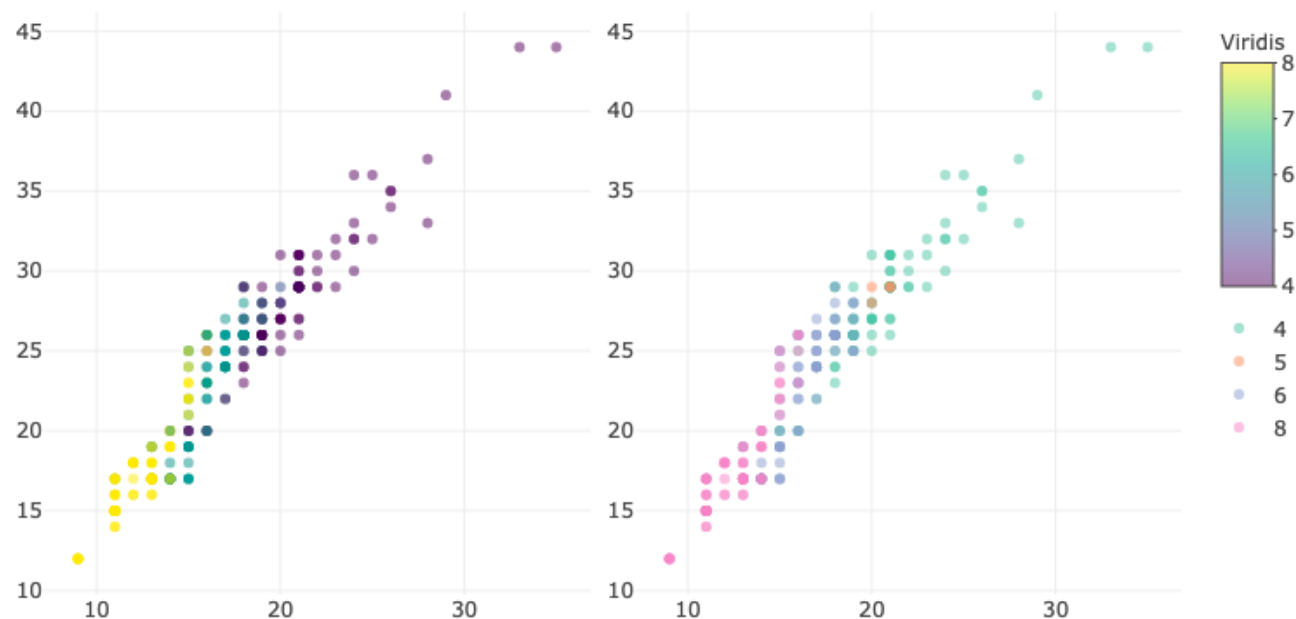
- **Alpha blending:** Visualizar “overplotting”:
 - `subplot(`
 - `plot_ly(mpg, x = ~cty, y = ~hwy, name = "default"),`
 - `plot_ly(mpg, x = ~cty, y = ~hwy) %>%`
 - `add_markers(alpha = 0.2, name = "alpha")`
 - `)`



2.- Creación de vistas: “scatter plots”

add_markers():

- **Manejo de colores:** *color*, *colors*, *colorbar()*, *hide_legend()*
 - `p <- plot_ly(mpg, x = ~cty, y = ~hwy, alpha = 0.5)`
 - `subplot(`
 - `add_markers(p, color = ~cyl, showlegend = FALSE) %>%`
 - `colorbar(title = "Viridis"),`
 - `add_markers(p, color = ~factor(cyl))`
 - `)`

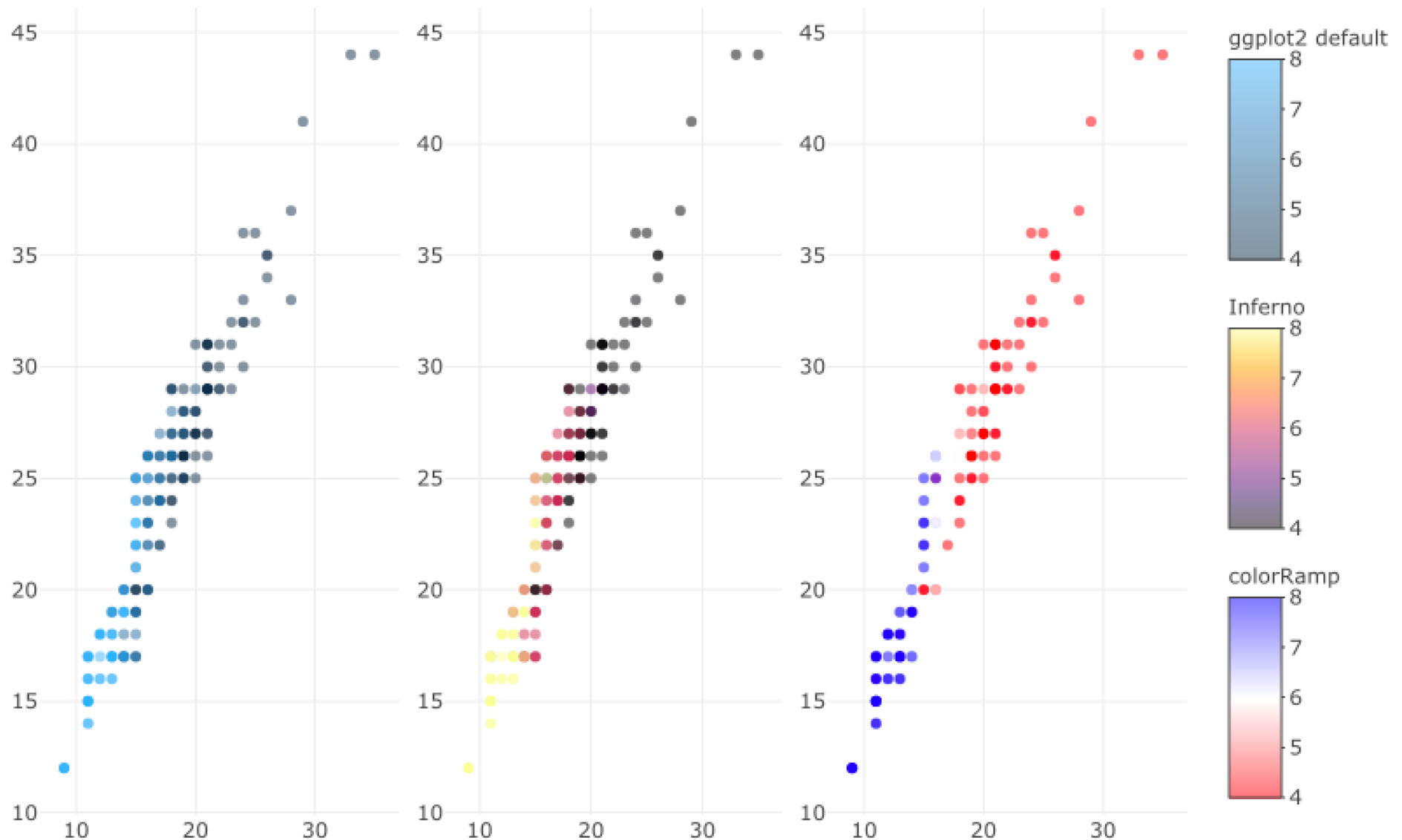


2.- Creación de vistas: “scatter plots”

add_markers():

- **Manejo de colores:** *color*, *colors*, *colorbar()*, *hide_legend()*
 - `col1 <- c("#132B43", "#56B1F7")`
 - `col2 <- viridisLite::inferno(10)`
 - `col3 <- colorRamp(c("red", "white", "blue"))`
 - `subplot(
 - add_markers(p, color = ~cyl, colors = col1) %>%
 - colorbar(title = "ggplot2 default"),
 - add_markers(p, color = ~cyl, colors = col2) %>%
 - colorbar(title = "Inferno"),
 - add_markers(p, color = ~cyl, colors = col3) %>%
 - colorbar(title = "colorRamp")
 -) %>% hide_legend()`

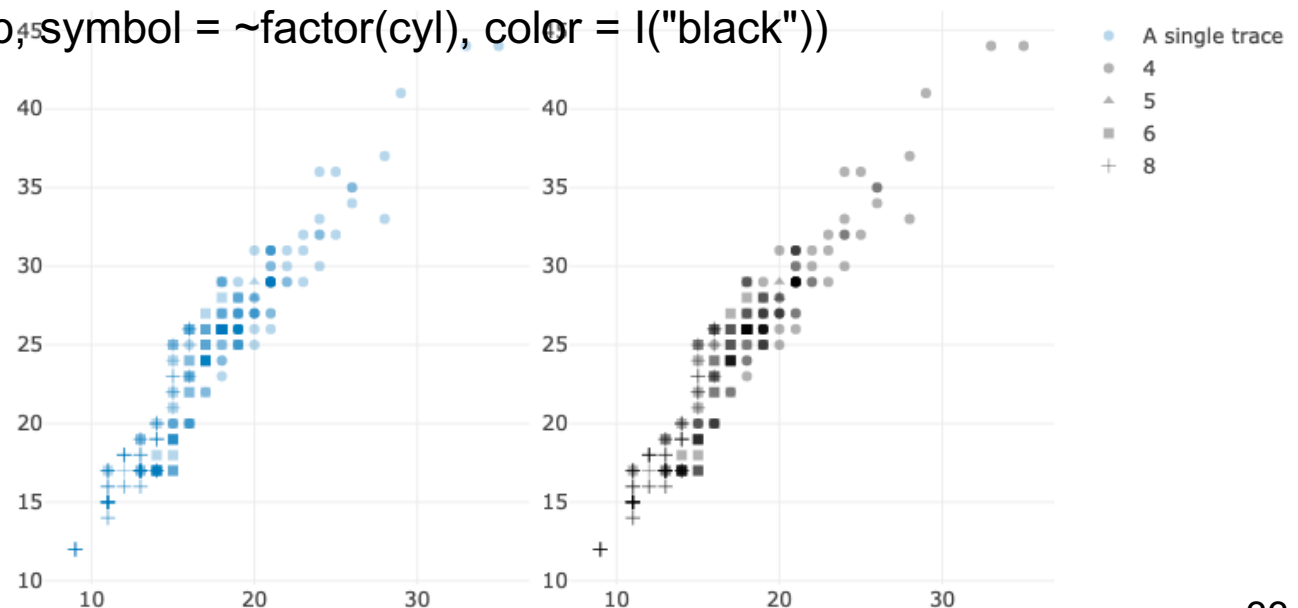
2.- Creación de vistas: “scatter plots”



2.- Creación de vistas: “scatter plots”

add_markers():

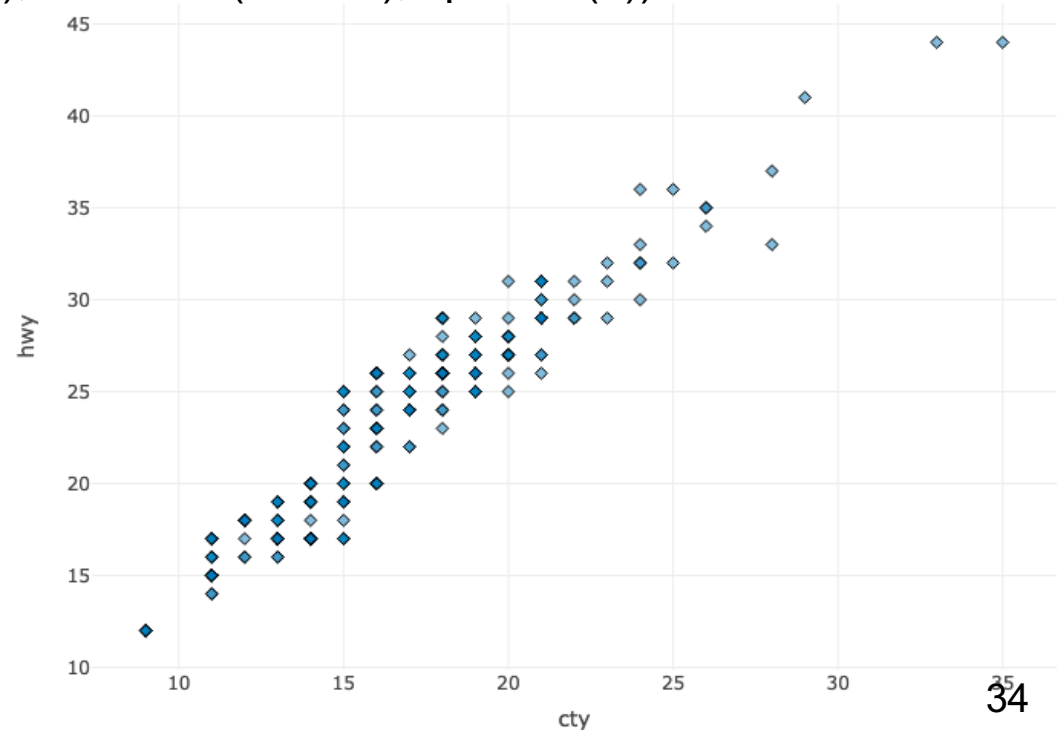
- **Símbolos:** *symbol*
 - Si es numérico genera una única traza.
 - Si no se especifica color, cada traza es de un color.
 - Se puede fijar un símbolo con *l()* o *c()*.
 - EJEMPLO:
 - `p <- plot_ly(mpg, x = ~cty, y = ~hwy, alpha = 0.3)`
 - `subplot(`
 - `add_markers(p, symbol = ~cyl, name = "A single trace"),`
 - `add_markers(p, symbol = ~factor(cyl), color = l("black"))`
 - `)`



2.- Creación de vistas: “scatter plots”

`add_markers()`:

- **Contornos:** *stroke*, *span*
 - Con *stroke* se indica el color del contorno.
 - Con *span* se fija el grosor de la línea.
 - Ambos esperan un atributo, para fijar valor *I()*.
 - EJEMPLO:
 - `plot_ly(mpg, x = ~cty, y = ~hwy, alpha = 0.5) %>%`
 - `add_markers(symbol = I(18), stroke = I("black"), span = I(1))`

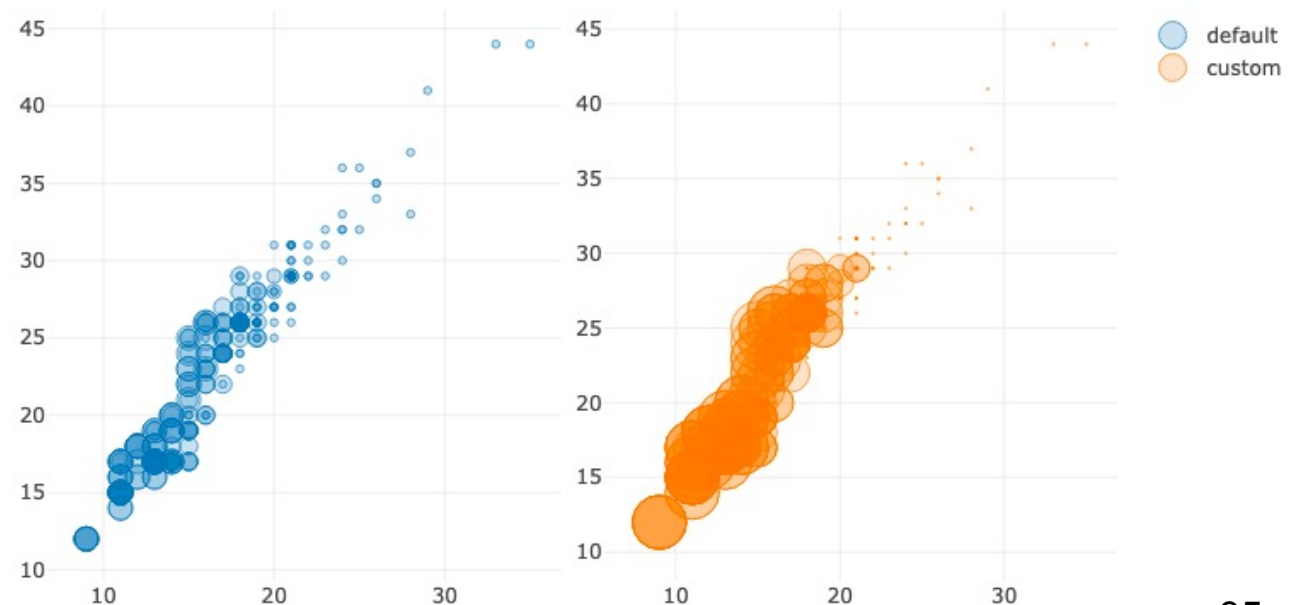


2.- Creación de vistas: “scatter plots”

add_markers():

- **Tamaño:** *size*, *sizes*

- Funciona de manera similar a *color* y *colors*.
- *c()* permite fijar un rango de tamaños con *sizes*.
- EJEMPLO:
 - `p <- plot_ly(mpg, x = ~cty, y = ~hwy, alpha = 0.3)`
 - `subplot(`
 - `add_markers(p, size = ~cyl, name = "default"),`
 - `add_markers(p, size = ~cyl, sizes = c(1, 500), name = "custom")`
 - `)`



2.- Creación de vistas: “scatter plots”

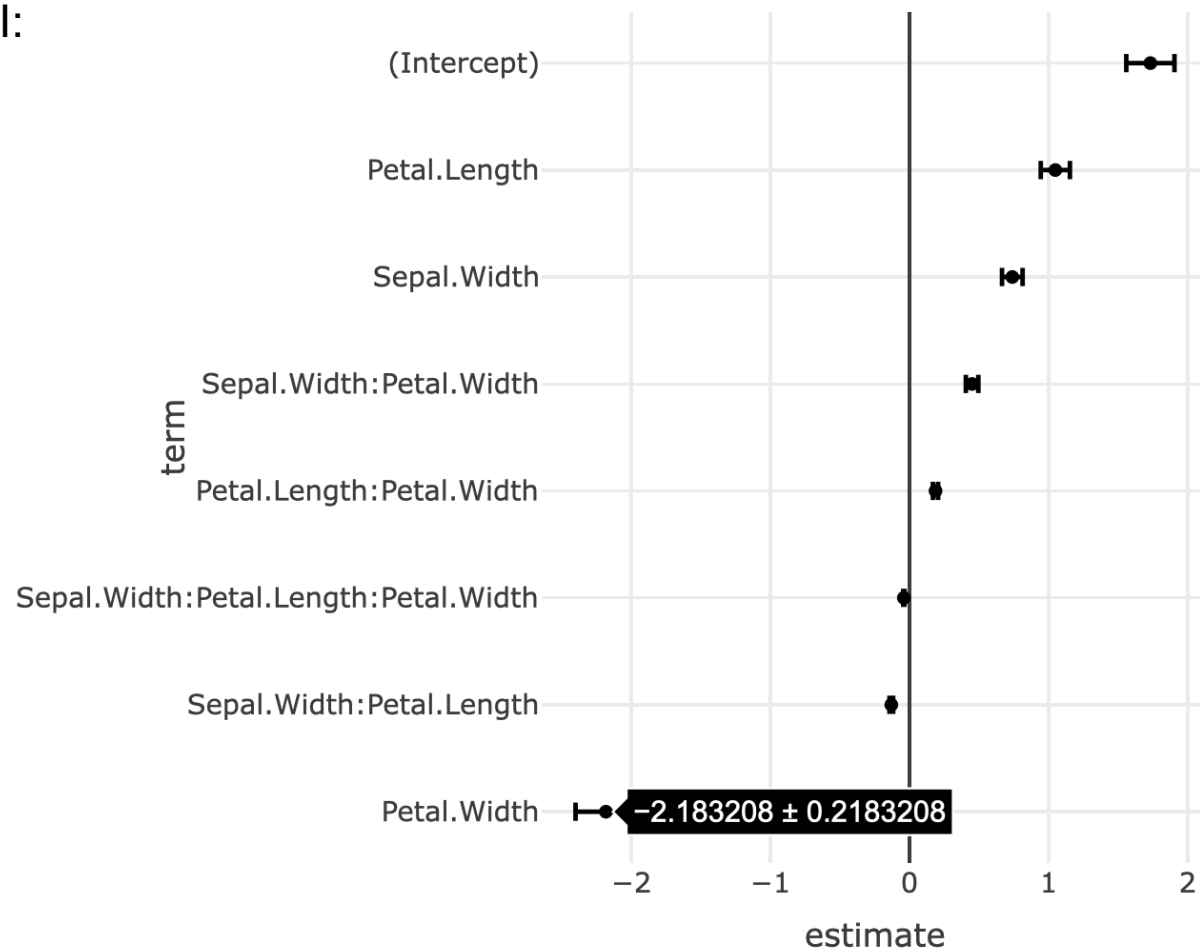
add_markers():

- **Dotplots** con barras de error.
 - En un Dotplot uno de los ejes es numérico y el otro categórico.
 - El objetivo es comparar valores por grupos.
 - EJEMPLO con XAI:
 - # Fit a full-factorial linear model
 - m <- lm(
 - Sepal.Length ~ Sepal.Width * Petal.Length * Petal.Width,
 - data = iris
 -)
 - broom::tidy(m) %>%
 - mutate(term = forcats::fct_reorder(term, estimate)) %>%
 - plot_ly(x = ~estimate, y = ~term) %>%
 - add_markers(
 - error_x = ~list(value = std.error),
 - color = I("black"),
 - hoverinfo = "x"
 -)

2.- Creación de vistas: “scatter plots”

`add_markers()`:

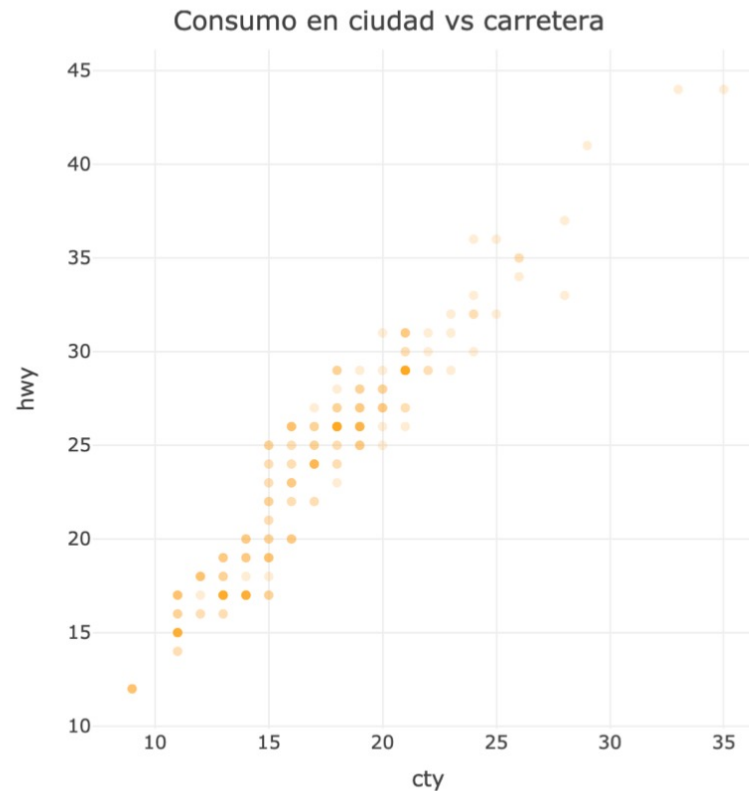
- **Dotplots** con barras de error.
 - En un Dotplot uno de los ejes es numérico y el otro categórico.
 - El objetivo es comparar valores por grupos.
 - EJEMPLO con XAI:



2.- Creación de vistas: “scatter plots”

add_markers():

- PRÁCTICA 1: add_markers() con plotly.



Need the use of “count()”

2.- Creación de vistas: “scatter plots”

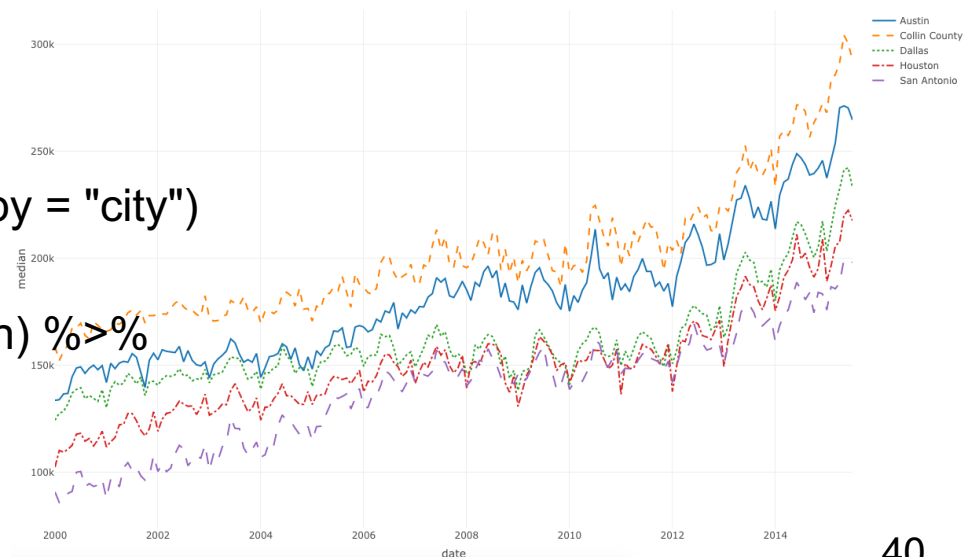
add_lines(), add_paths(), add_segments():

- **add_lines():** genera líneas entre puntos consecutivos. Antes de generar las líneas ordena los puntos por valor “x”.
- **add_paths():** genera líneas entre puntos consecutivos. No hace ordenamiento previo.
- **add_segments():** genera líneas entre pares de puntos (x, y)-(xend, yend)

2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- **Tipos de líneas:** *linetype*, *linetypes*.
 - Funciona de manera similar a *color* y *colors*.
 - *c()* permite asignar tipos de líneas a trazas (valores discretos de un atributo).
 - Tipos: “dashdot”, “longdash”, “dash”, “solid”, “dot”.
 - EJEMPLO con *add_lines()*:
 - `library(dplyr)`
 - `top5 <- txhousing %>%`
 - `group_by(city) %>%`
 - `summarise(m = mean(sales, na.rm = TRUE)) %>%`
 - `arrange(desc(m)) %>%`
 - `top_n(5)`
 - `tx5 <- semi_join(txhousing, top5, by = "city")`
 - `plot_ly(tx5, x = ~date, y = ~median) %>%`
 - `add_lines(linetype = ~city)`



2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- **Gráfico de inclinación (slopegraph):**

- Permite analizar variaciones de valores.
- Se genera con *add_segments()*.
- EJEMPLO:

- `pru <- mpg %>% group_by(manufacturer) %>% summarise(cty_m = mean(cty), hwy_m = mean(hwy)) %>%`

- `plot_ly() %>% add_segments( x = 1, xend = 2, y = ~cty_m, yend = ~hwy_m, color = l("gray90") ) %>%`

- `add_annotations(x = 1, y = ~cty_m, text = ~manufacturer, xanchor = "right", showarrow = FALSE) %>%`

- `add_annotations(x = 2, y = ~hwy_m, text = ~manufacturer, xanchor = "left", showarrow = FALSE ) %>%`

- `Layout(title = "Consumo...", showlegend = FALSE, xaxis = list( range = c(0, 3), ticktext = c("Ciudad", "Carretera"), tickvals = c(1, 2), zeroline = FALSE), yaxis = list( title = "", showgrid = FALSE, showticks = FALSE, showticklabels = FALSE)) %>%`

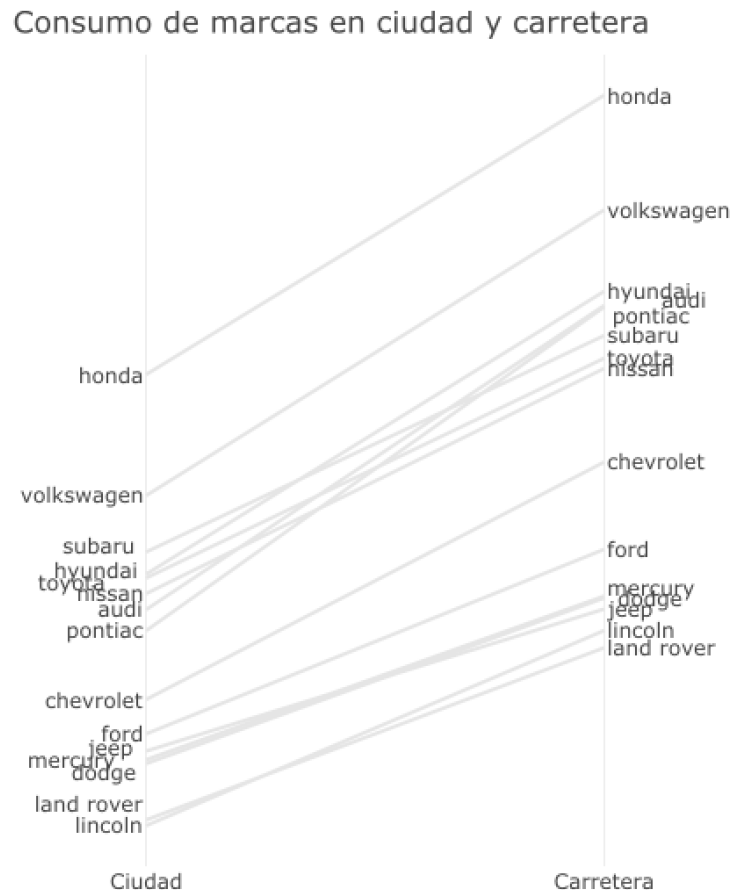
- `config(edits = list(annotationPosition = TRUE))`

2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- **Gráfico de inclinación (slopegraph):**

- Permite analizar variaciones de valores.
- Se genera con *add_segments()*.
- EJEMPLO:



2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- **Parallel coordinates:**

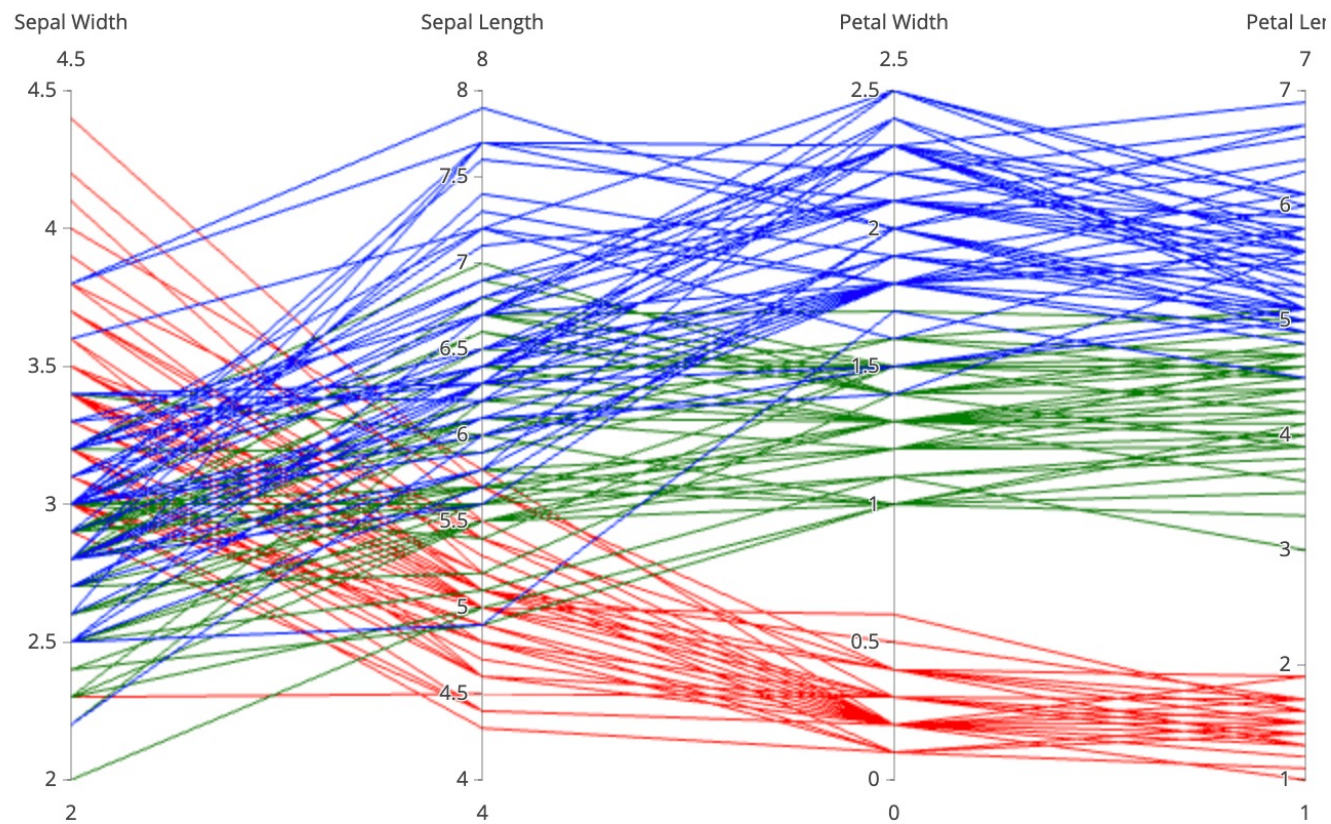
- EJEMPLO (Iris dataset):

- iris %>% plot_ly(type = 'parcoords',
 - line = list(color = ~species_id,
 - colorscale = list(c(0,'red'),c(0.5,'green'),c(1,'blue'))),
 - dimensions = list(
 - list(range = c(2,4.5),
 - label = 'Sepal Width', values = ~sepal_width),
 - list(range = c(4,8),
 - label = 'Sepal Length', values = ~sepal_length),
 - list(range = c(0,2.5),
 - label = 'Petal Width', values = ~petal_width),
 - list(range = c(1,7),
 - label = 'Petal Length', values = ~petal_length)
 -)
 -)

2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- **Parallel coordinates:**
 - EJEMPLO (Iris dataset):



2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

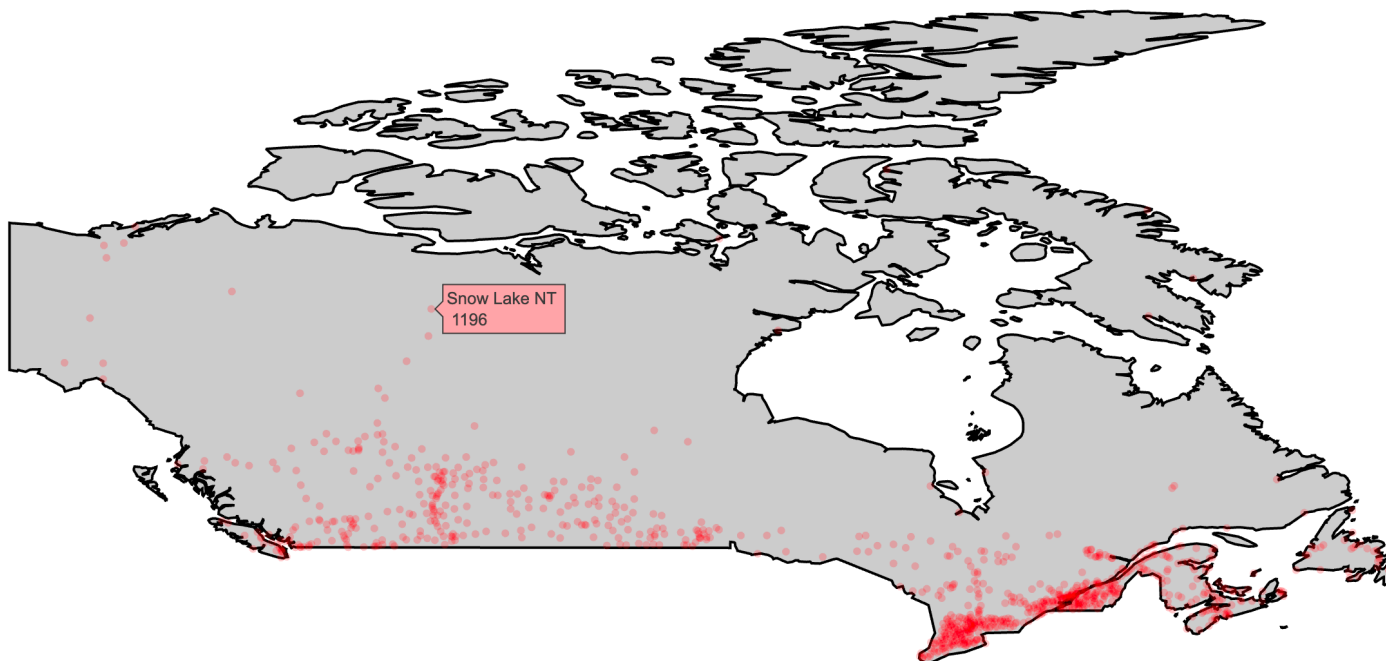
- ***add_polygons()*:**

- Similar a *add_paths()* (i.e. genera líneas sin ordenar) pero con el área rellena.
- Útil para dibujar mapas.
- Se debe fijar el ratio de aspecto (*scaleanchor* = “y”)
- EJEMPLO (mapa de Canadá):
 - `base <- map_data("world", "canada") %>%`
 - `group_by(group) %>%`
 - `plotly_empty(x = ~long, y = ~lat, alpha = 0.2) %>%`
 - `layout(showlegend = FALSE, xaxis = list(scaleanchor = "y"))`

2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- ***add_polygons():***
 - base %>%
 - add_polygons(hoverinfo = "none", color = I("black")) %>%
 - add_markers(text = ~paste(name, "
", pop), hoverinfo = "text",
 - color = I("red"), data = maps::canada.cities)

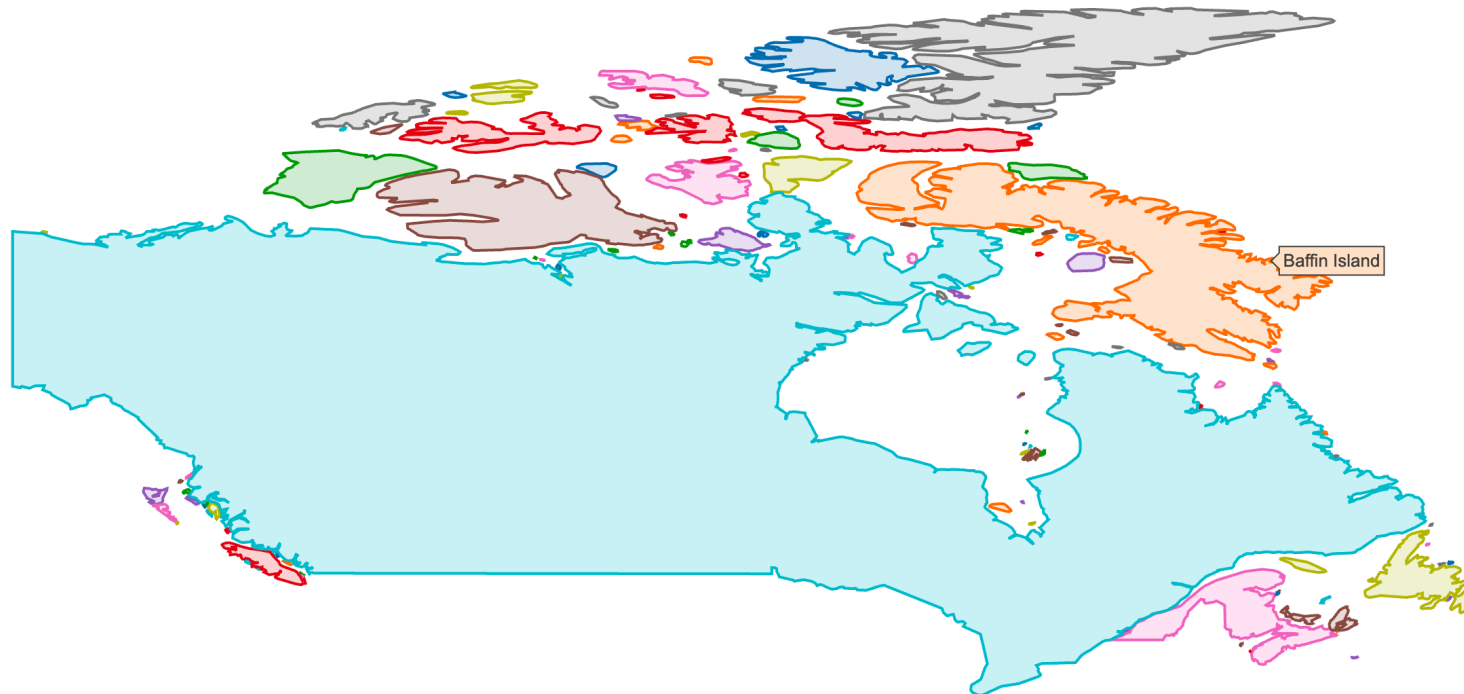


2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- ***add_polygons():***

- base %>%
- add_polygons(base, split = ~subregion, hoveron = "fills")



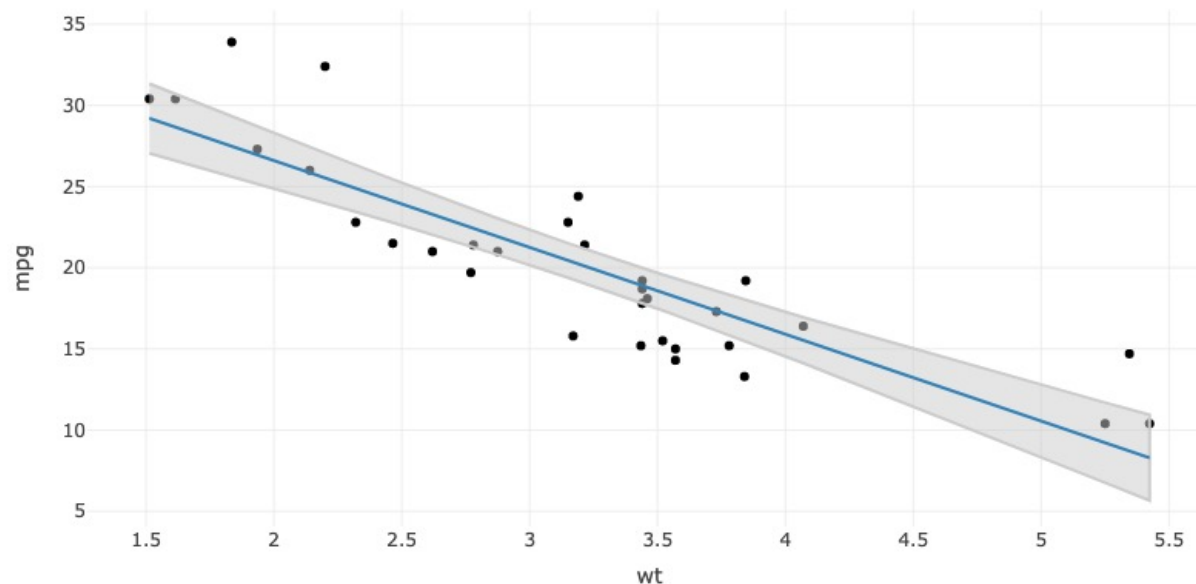
2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- **add_ribbons():**

- Muestra la incerteza de una función por cada valor x.
- Parámetros: *x*, *ymin*, *ymax*.
- EJEMPLO:

- `m <- lm(mpg ~ wt, data = mtcars)`
- `broom::augment(m) %>% plot_ly(x = ~wt, showlegend = FALSE) %>%`
- `add_markers(y = ~mpg, color = I("black")) %>%`
- `add_ribbons(ymin = ~.fitted - 1.96 * .se.fit, ymax = ~.fitted + 1.96 * .se.fit,`
- `color = I("gray80")) %>% add_lines(y = ~.fitted, color = I("steelblue"))`

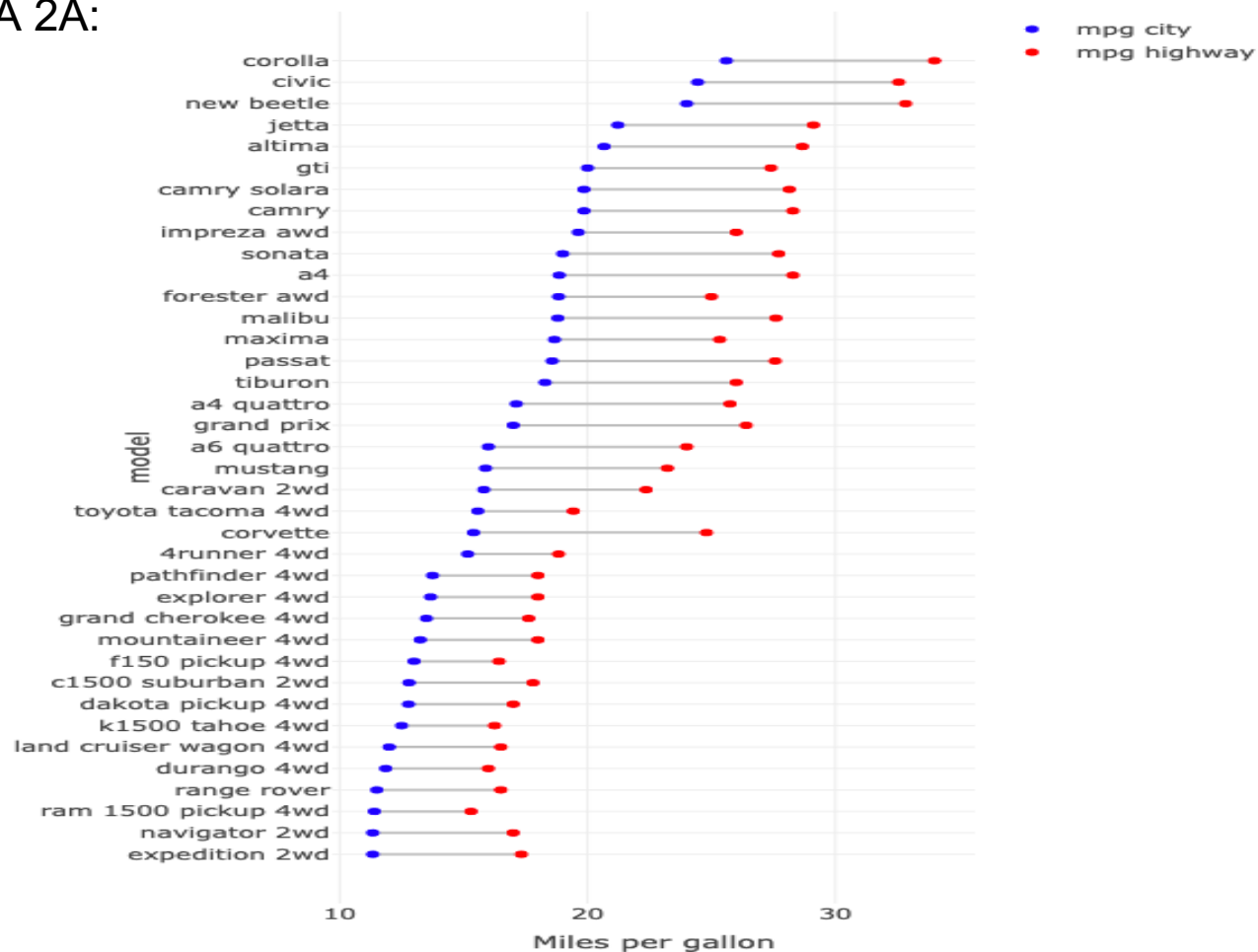


2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- **Dumbbell:**

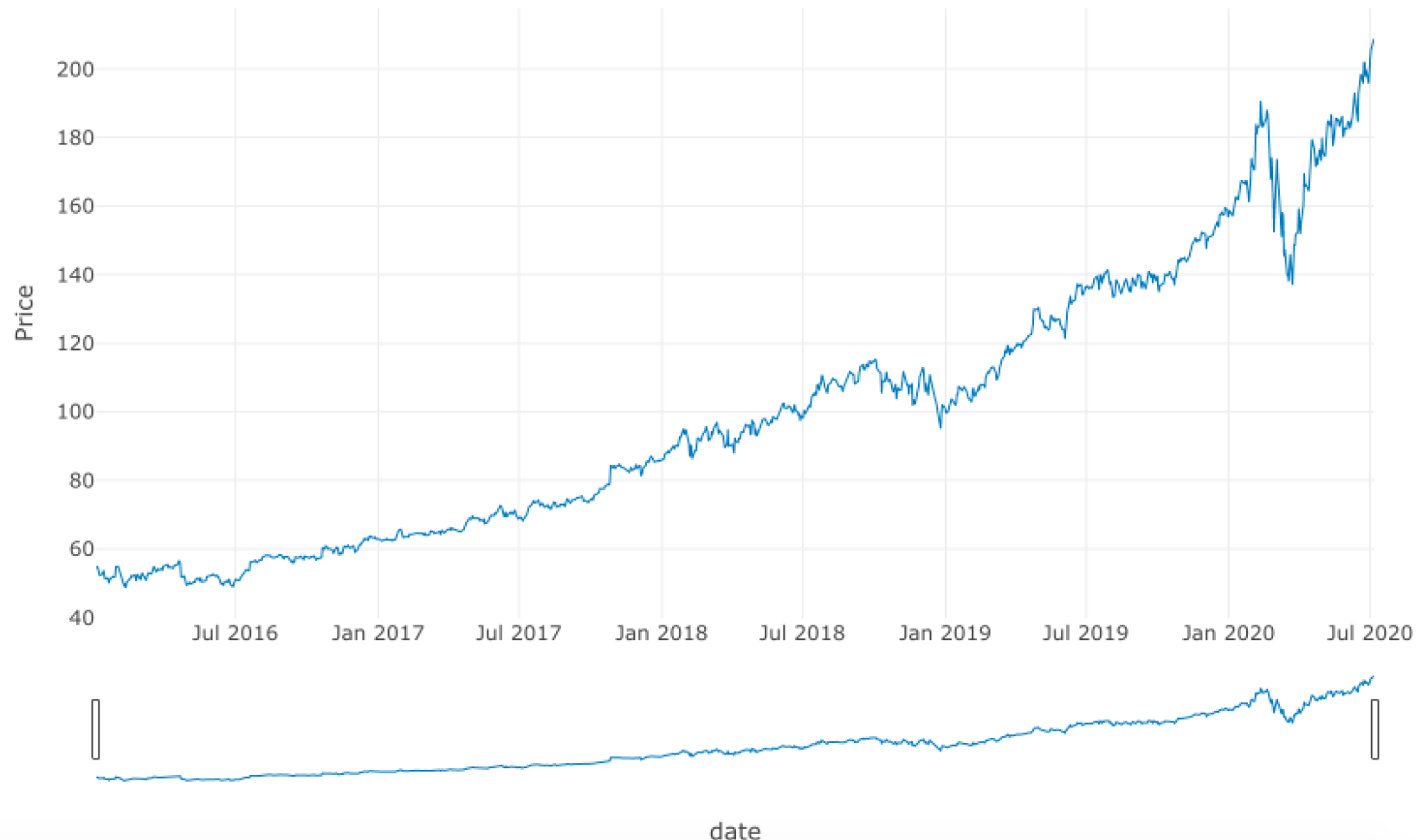
- Comparar dos clases de valores numéricos de muchos grupos.
- Se genera con *add_segments()*.
- PRACTICA 2A:



2.- Creación de vistas: “scatter plots”

add_lines(), add_paths(), add_segments():

- ***rangeslider()***:
 - PRACTICA 2B (evolución de las acciones de Microsoft):



2.- Creación de vistas: mapas

- Dos métodos de generar mapas:
 - Utilizando métodos integrados (rapidez pero poco control).
 - Utilizando métodos personalizados (más complejo pero más control)
- Si se observan limitaciones en **plotly**, se puede usar (incluso combinar) con: leaflet, mapview, mapedit, tmap, mapdeck, ...

2.- Creación de vistas: mapas

- Dos métodos de generar mapas:
 - Utilizando métodos integrados (rapidez pero poco control).
 - Utilizando métodos personalizados (más complejo pero más control)

2.- Creación de vistas: mapas

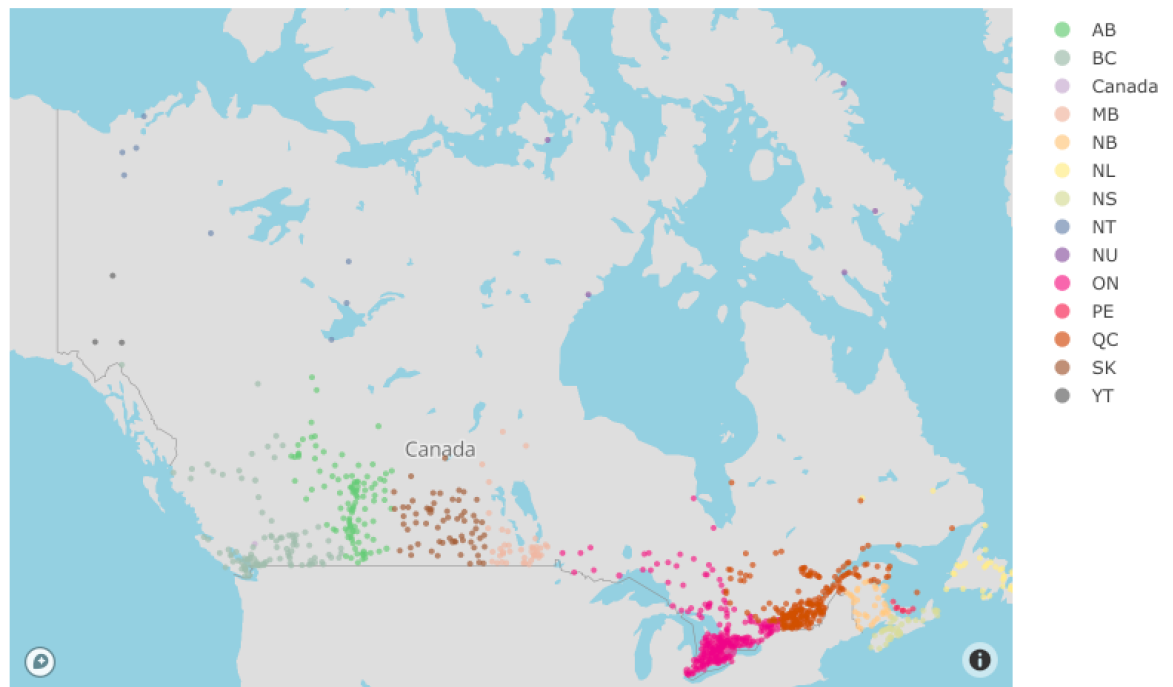
plot_mapbox():

- Requiere darse de alta en **mapbox**.
- Una vez realizada el alta, proporcionan un código con el que instanciar MAXBOX_TOKEN:
 - `Sys_setenv(MAPBOX_TOKEN = “.....”)`
 - Se puede combinar con cualquier “scatter plot”.
 - Tipos de mapas:
 - “basic”
 - “outdoors”
 - “satellite-streets”
 - “white-bg”
 - “carto-darkmatter”
 - “stamen-toner”
 - “streets”
 - “light”
 - “satellite”
 - “open-street-map”
 - “carto-positron”
 - “stamen-terrain”
 - “stamen-watercolor”

2.- Creación de vistas: mapas

plot_mapbox():

- EJEMPLO:
 - `plot_mapbox(maps::canada.cities) %>%`
 - `add_markers(x = ~long, y = ~lat, size = ~pop, color = ~country.etc, colors = "Accent", text = ~paste(name, pop), hoverinfo = "text")`



2.- Creación de vistas: mapas

plot_mapbox():

- EJEMPLO instanciación de tipo de mapa:
 - `plot_mapbox() %>% layout(mapbox = list(style = "satellite"))`



2.- Creación de vistas: mapas

plot_geo():

- Tiene más tipos de proyecciones que **mapbox**.
- Los mapas son más limitados.
 - Tipos de proyecciones:
 - 'equirectangular'
 - 'mercator'
 - 'orthographic'
 - 'natural earth'
 - 'kavrayskiy7'
 - 'miller'
 - 'robinson'
 - 'eckert4'
 - 'azimuthal equal area'
 - 'azimuthal equidistant'
 - 'conic equal area'
 - 'conic conformal'
 - 'conic equidistant'
 - 'gnomonic'
 - 'stereographic'
 - 'mollweide'
 - 'hammer'
 - 'transverse mercator'
 - 'albers usa'

2.- Creación de vistas: mapas

plot_geo():

- EJEMPLO: (vuelos USA)

- # map projection

- `geo <- list(projection = list(type = 'orthographic',
rotation = list(lon = -100, lat = 40, roll = 0)), showland = TRUE,
landcolor = toRGB("gray95"))`

- `plot_geo(color = l("red")) %>%`

- `add_markers(data = air, x = ~long, y = ~lat, text = ~airport,
size = ~cnt, hoverinfo = "text", alpha = 0.5) %>%`

- `add_segments(data = flights,x = ~start_lon, xend = ~end_lon, y =
~start_lat, yend = ~end_lat, alpha = 0.3, size = l(1),
hoverinfo = "none") %>%`

- `layout(geo = geo, showlegend = FALSE)`

2.- Creación de vistas: mapas

plot_geo():

- EJEMPLO: (vuelos USA)

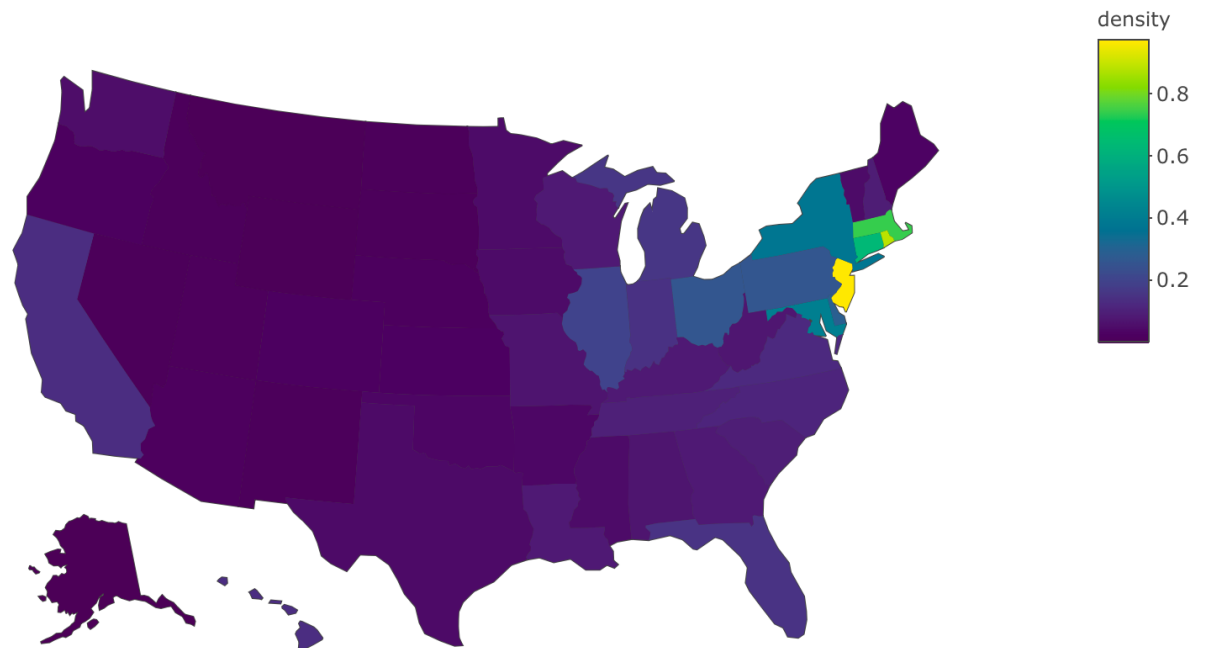


2.- Creación de vistas: Choropleths

- Los mapas coropléticos (**choropleths**) permiten visualizar información geográfica sobre un mapa de colores.
- Tanto *plot_geo()* como *plot_mapbox()* están optimizados para coropletos.
- Parámetros a instanciar:
 - *Z* -> se asignará un atributo que determinará el color.
 - *locations* -> establece coordenadas mediante identificadores o nombres de función.
 - *locationmode* -> Determina el conjunto de ubicaciones utilizadas para hacer coincidir las entradas en "ubicaciones" con las regiones del mapa. Los valores: "ISO-3", "USA-states", "country names", "geojson-id".

2.- Creación de vistas: Choropleths

- EJEMPLO:
 - `density <- state.x77[, "Population"] / state.x77[, "Area"]`
 - `g <- list(scope = 'usa', projection = list(type = 'albers usa'))`
 - `plot_geo() %>%`
 `add_trace(`
 `z = ~density, text = state.name, span = l(0),`
 `locations = state.abb, locationmode = 'USA-states'`
 `) %>%`
 `layout(geo = g)`



2.- Creación de vistas: mapas

- Dos métodos de generar mapas:
 - Utilizando métodos integrados (rapidez pero poco control).
 - Utilizando métodos personalizados (más complejo pero más control)

2.- Creación de vistas: mapas

Simple Features (sf):

- Paquete para trabajar con información geo-espacial.
- La información geo-espacial la almacena en una columna de listas.
- Posee únicamente una fila por “dato” observado.
- Paquetes con información **sf**: `rnaturalearth`, `USAboundaries`, `rnaturalearthhires`, ...
- Cuando **plotly** recibe un objeto **sf** ya sabe cómo tratarlo.

2.- Creación de vistas: mapas

EJEMPLO:

- `library(rnaturalearth)`
- `world <- ne_countries(returnclass = "sf")`
- `plot_ly(world, color = I("gray90"), stroke = I("black"), span = I(1))`



2.- Creación de vistas: mapas

Contenido estructura sf:

- world %>% select(name) %>% print(n=4)

```
Simple feature collection with 177 features and 1 field
geometry type:  MULTIPOLYGON
dimension:      XY
bbox:           xmin: -180 ymin: -90 xmax: 180 ymax: 83.64513
epsg (SRID):    4326
proj4string:    +proj=longlat +datum=WGS84 +no_defs
First 4 features:
```

	name	geometry
0	Afghanistan	MULTIPOLYGON (((61.21082 35...
1	Angola	MULTIPOLYGON (((16.32653 -5...
2	Albania	MULTIPOLYGON (((20.59025 41...
3	United Arab Emirates	MULTIPOLYGON (((51.57952 24...

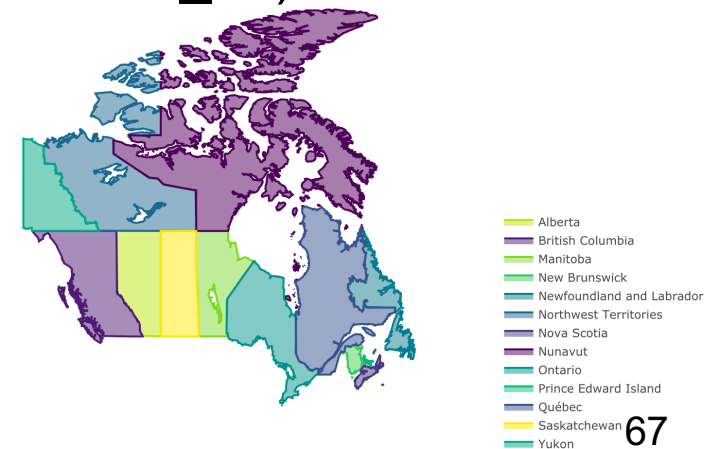
2.- Creación de vistas: mapas

Simple Features (sf) en plotly:

- Lo dibuja todo en una única traza (trace/capa).
- Si se desea que cada polígono sea una traza (para dibujar cada uno de un color) usar el parámetro *split* con un atributo con valor único por traza.

EJEMPLO:

- `canada <- ne_states(country = "Canada", returnclass = "sf")`
- `plot_ly(canada, split = ~name, color = ~provnum_ne)`



2.- Creación de vistas: mapas

Simple Features (sf) en plotly:

- Para generar texto adaptado utilizar: *text*, *hoveron*="fills", *hoverinfo*="text".
- Otras proyecciones y usos: <http://spatialreference.org/>

EJEMPLO:

- ```
plot_ly(canada, split = ~name, color = l("gray90"),
 text = ~paste(name, "is \n province number", provnum_ne),
 hoveron = "fills", hoverinfo = "text",
 showlegend = FALSE
)
```

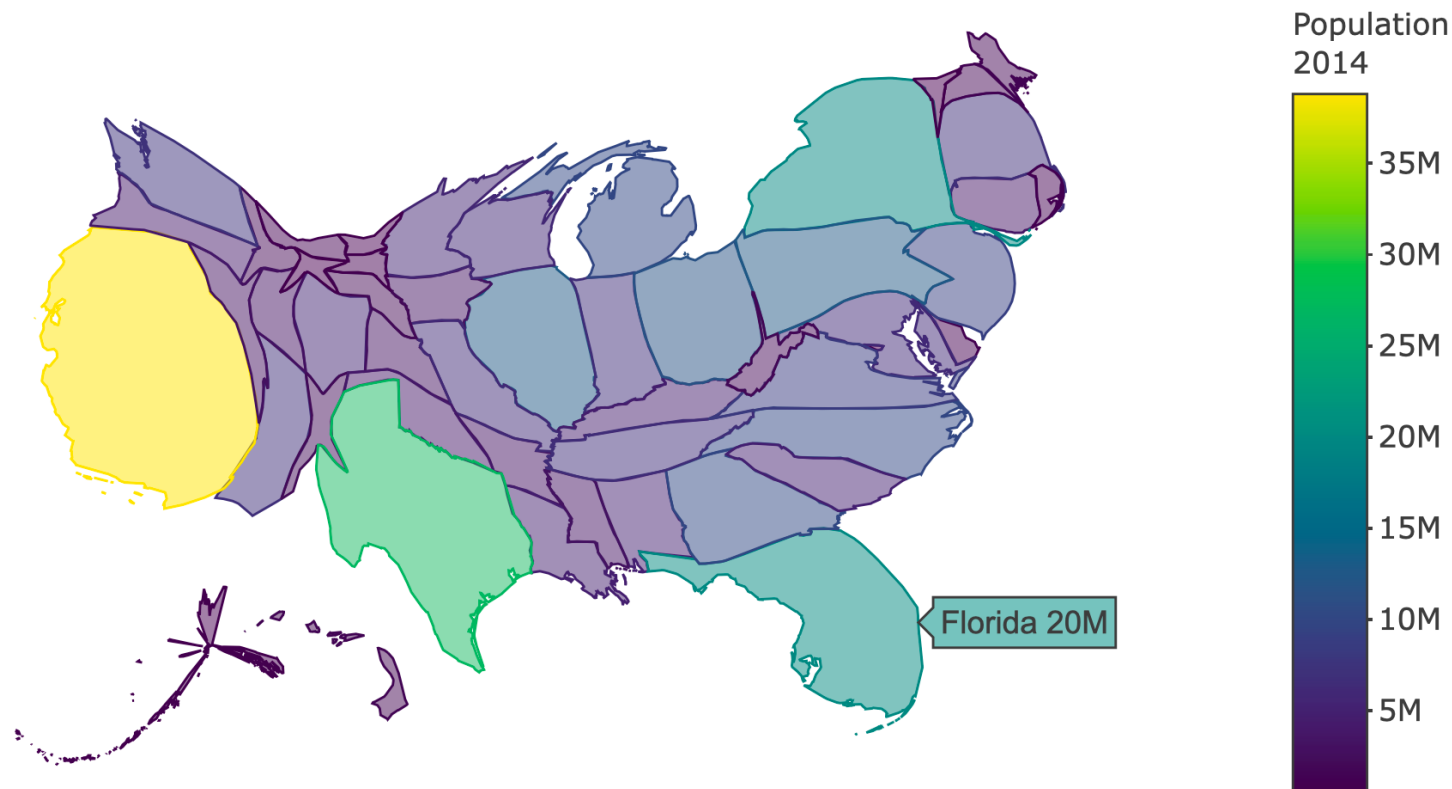


## 2.- Creación de vistas: mapas

---

OTROS TIPOS DE MAPAS: (librería **cartogram**).

- *cartogram\_cont()*

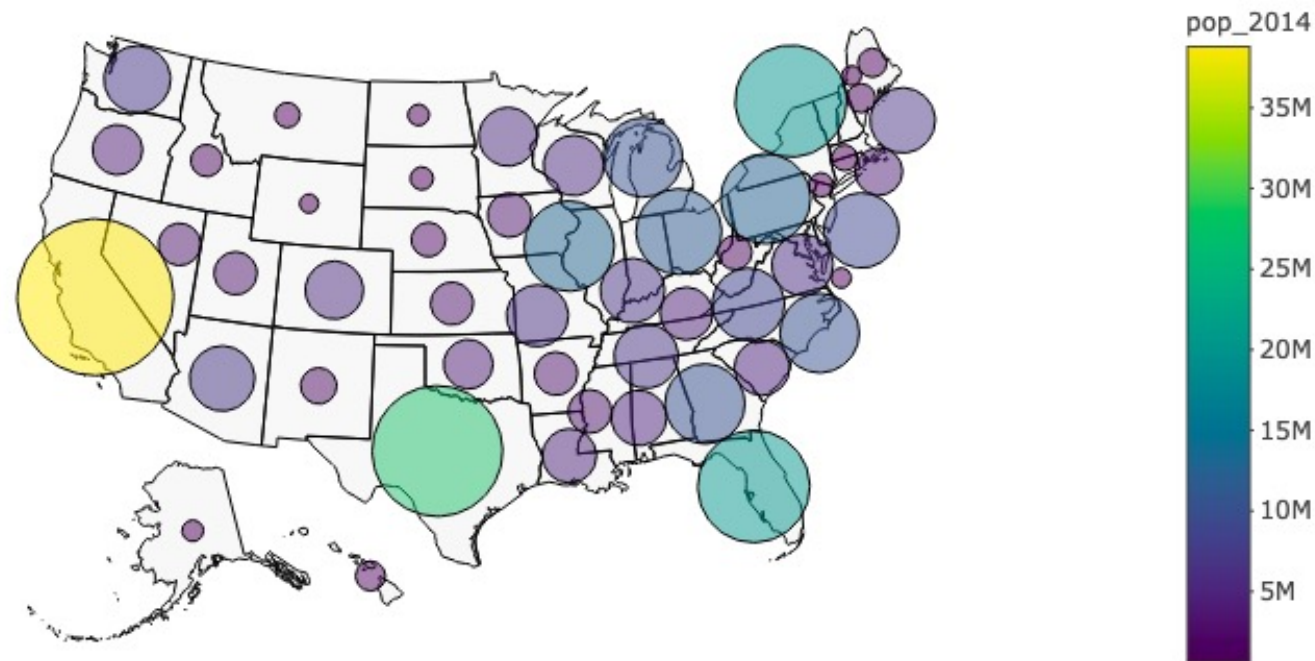


## 2.- Creación de vistas: mapas

---

OTROS TIPOS DE MAPAS: (librería **cartogram**).

- *cartogram\_dorling()*

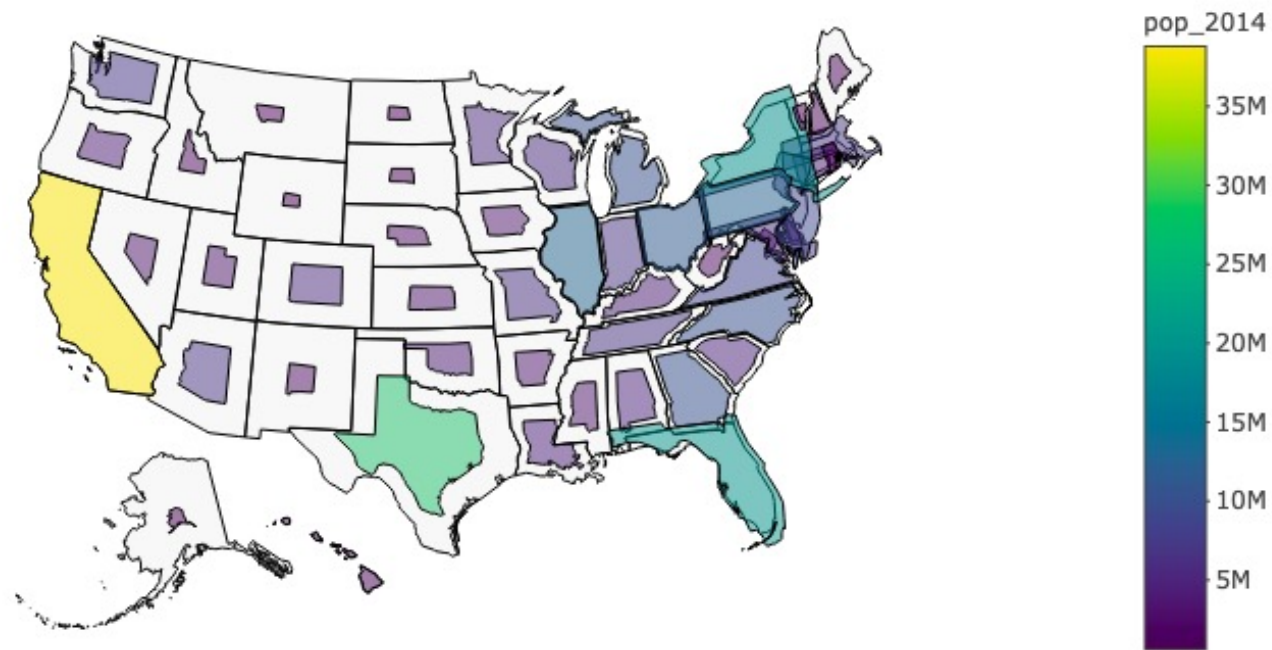


## 2.- Creación de vistas: mapas

---

OTROS TIPOS DE MAPAS: (librería **cartogram**).

- *cartogram\_ncont()*





## 2.- Creación de vistas: histogramas y barras

---

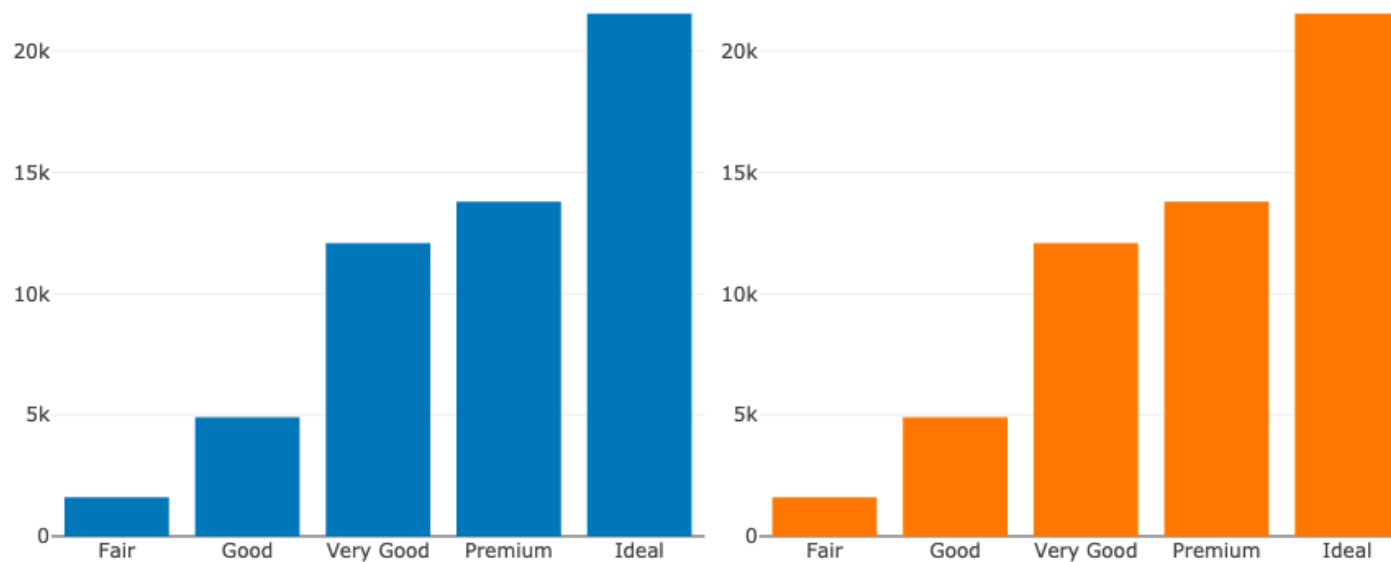
- `add_histogram()` los contenedores o “bins” son calculados por **plotly**.
  - Sólo requiere una variable.
- `addBars()` todos los cálculos son realizados por el usuario.
  - *Requiere dos variables.*
- Parámetros:
  - `x`: datos
  - `y`: frecuencia absoluta o relativa (sólo *bars*)
  - `xbins/ybins`: personalización del número de contenedores (sólo *histogram*).

## 2.- Creación de vistas: histogramas y barras

---

### EJEMPLO:

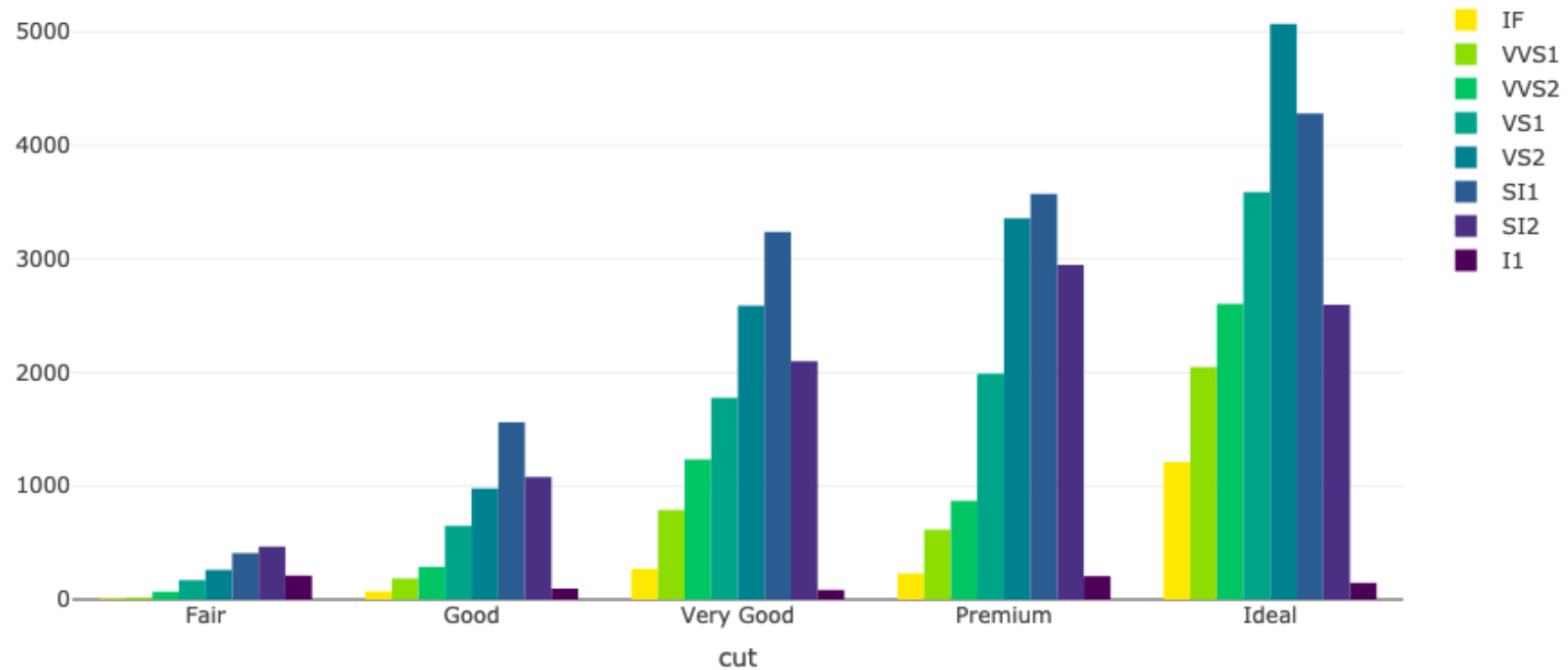
- `library(dplyr)`
- `p1 <- plot_ly(diamonds, x = ~cut) %>% add_histogram()`
- `p2 <- diamonds %>% count(cut) %>% plot_ly(x = ~cut, y = ~n) %>% add_bars()`
- `subplot(p1, p2) %>% hide_legend()`



## 2.- Creación de vistas: histogramas y barras

### EJEMPLO (múltiples distribuciones):

– `plot_ly(diamonds, x = ~cut, color = ~clarity) %>% add_histogram()`

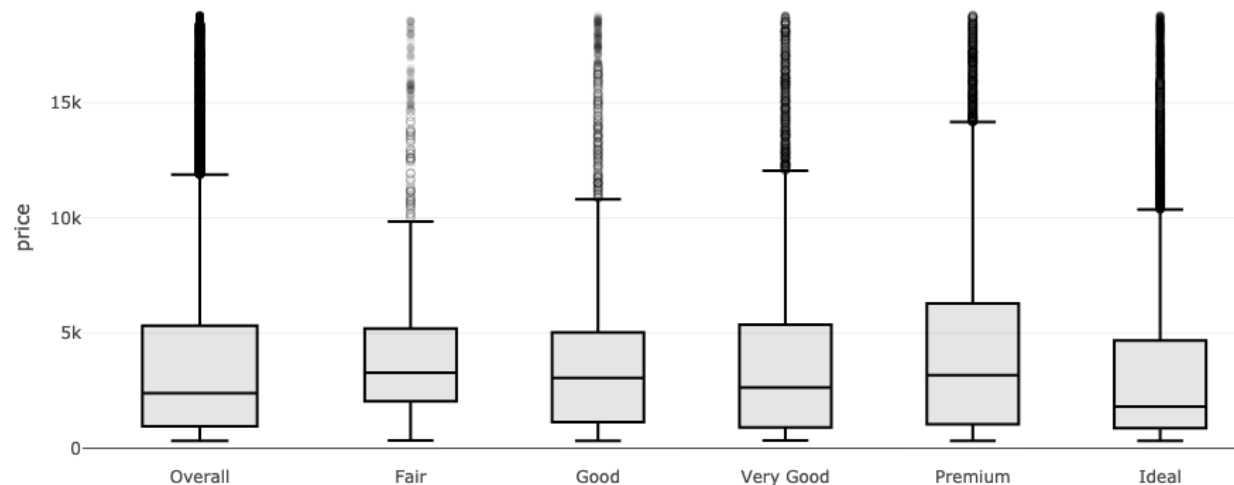


## 2.- Creación de vistas: boxplots

- `add_boxplot()`
  - Requiere un atributo numérico y otro categórico.
  - Orientación dependiendo si se proporciona a través de x o y.

### EJEMPLO:

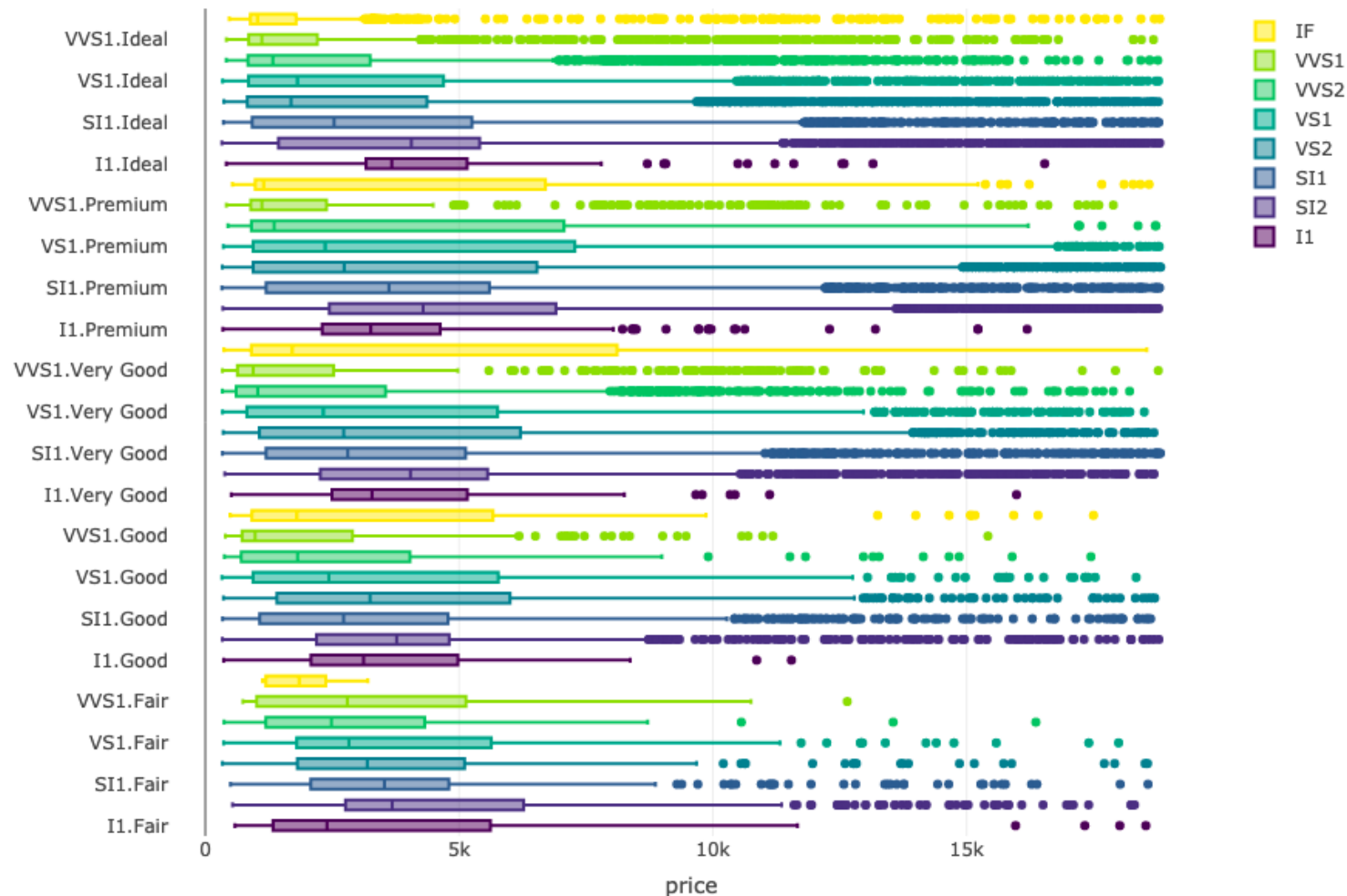
- `p <- plot_ly(diamonds, y = ~price, color = I("black"), alpha = 0.1, boxpoints = "suspectedoutliers")`
- `p1 <- p %>% add_boxplot(x = "Overall")`
- `p2 <- p %>% add_boxplot(x = ~cut)`
- `subplot(p1, p2, shareY = TRUE, widths = c(0.2, 0.8), margin = 0) %>% hide_legend()`



## 2.- Creación de vistas: boxplots

### EJEMPLO:

- `plot_ly(diamonds, x = ~price, y = ~interaction(clarity, cut)) %>%  
 add_boxplot(color = ~clarity) %>% layout(yaxis = list(title = ""))`



## 2.- Creación de vistas: frecuencias 2D

---

- *add\_heatmap()*
  - Funciona como *add\_bars()* pero en 2D.
  - Hay que proporcionar los datos procesados.
- *add\_histogram2d()*
  - Funciona como *add\_histogram()* pero en 2D
  - Los cálculos los realiza **plotly**.
- Parámetros:
  - *x*: atributo 1 numérico o categórico.
  - *y*: atributo 2 numérico o categórico.
  - *zs-smooth*: determinación automática del número de “bins”.
  - *nbinsx/nbinsy*: atributo para fijar en el número de “bins”.

## 2.- Creación de vistas: frecuencias 2D

- EJEMPLO “histogram”:

- `p <- plot_ly(diamonds, x = ~log(carat), y = ~log(price))`

- `subplot(`

- `add_histogram2d(p) %>%`

- `colorbar(title = "default") %>%`

- `layout(xaxis = list(title = "default")),`

- `add_histogram2d(p, zsmooth = "best") %>%`

- `colorbar(title = "zsmooth") %>%`

- `layout(xaxis = list(title = "zsmooth")),`

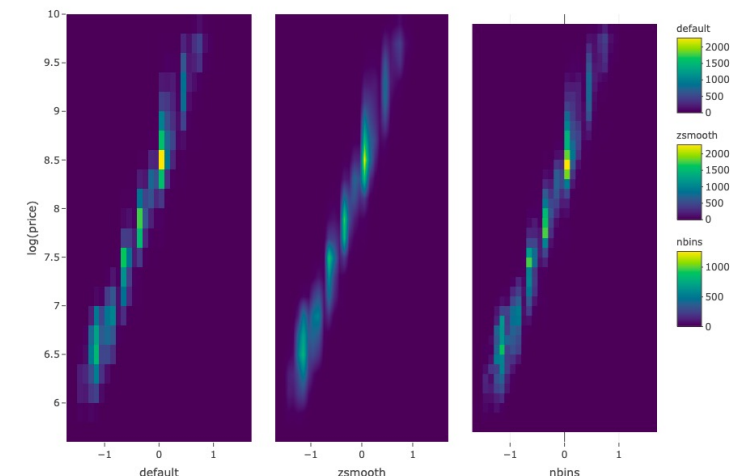
- `add_histogram2d(p, nbinsx = 60, nbinsy = 60) %>%`

- `colorbar(title = "nbins") %>%`

- `layout(xaxis = list(title = "nbins")),`

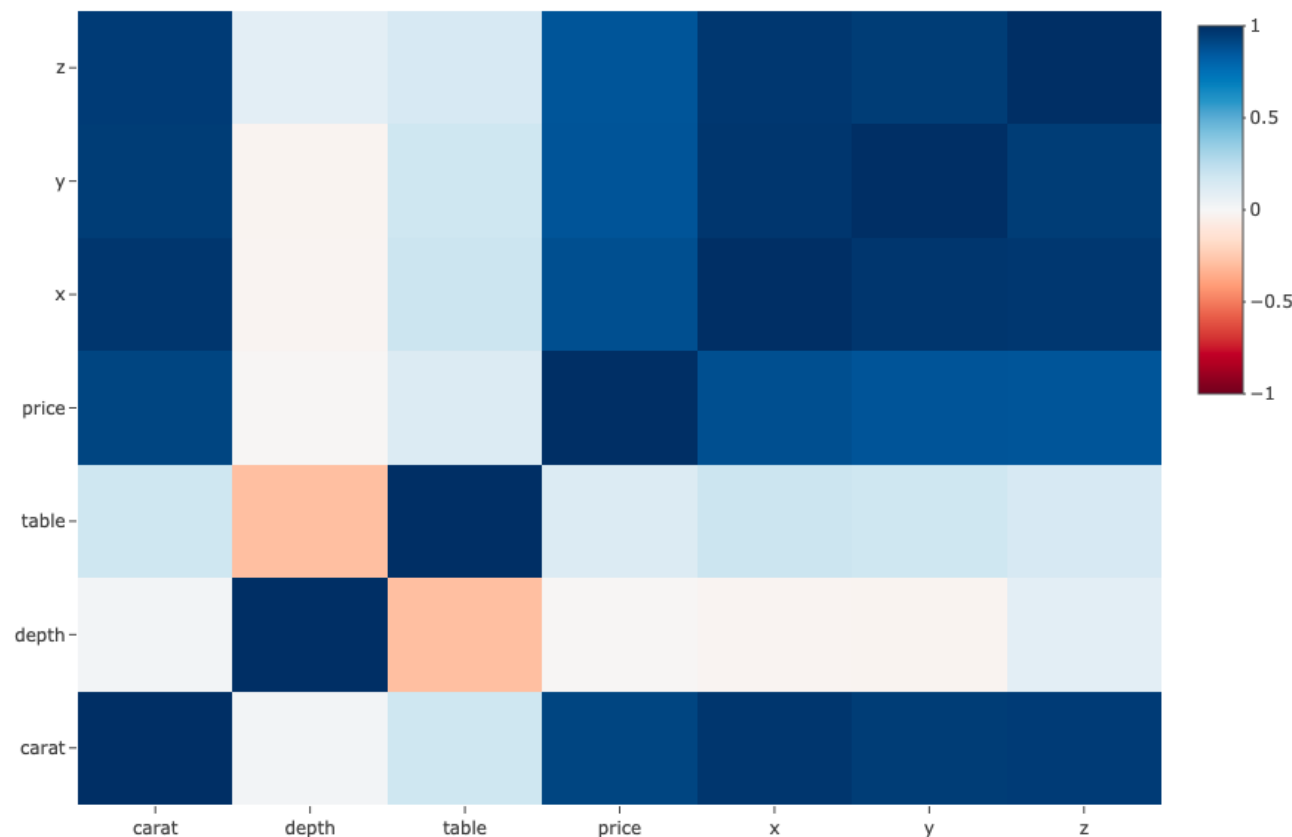
- `shareY = TRUE, titleX = TRUE`

- `)`



## 2.- Creación de vistas: frecuencias 2D

- EJEMPLO "heatmap":
  - `corr <- cor(dplyr::select_if(diamonds, is.numeric))`
  - `plot_ly(colors = "RdBu") %>%  
 add_heatmap(x = rownames(corr), y = colnames(corr), z = corr)  
 %>% colorbar(limits = c(-1, 1))`



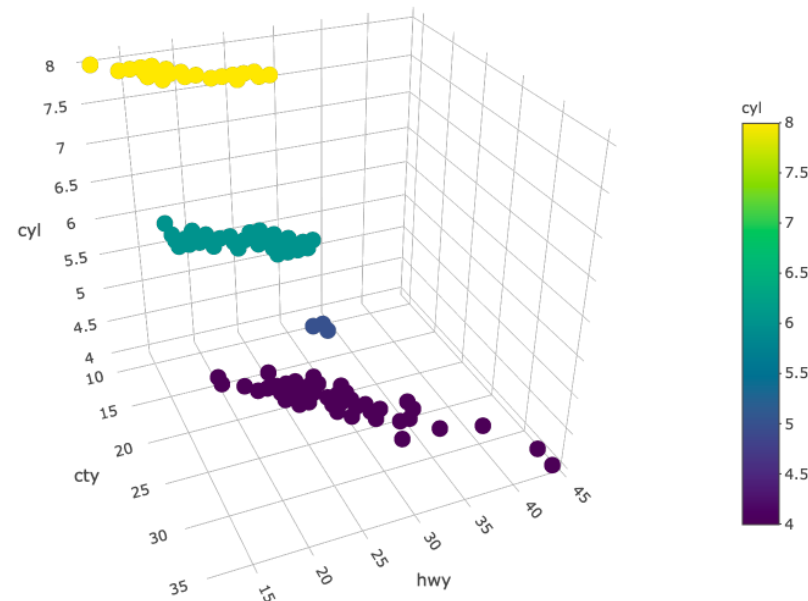


## 2.- Creación de vistas: gráficos 3D

- **Marcadores (*add\_markers*), caminos (*add\_paths*) y líneas (*add\_lines*):**
  - Basta con añadir el parámetro *z* al que se le asocia un atributo en *plot\_ly()*.

### EJEMPLO:

- `plot_ly(mpg, x = ~cty, y = ~hwy, z = ~cyl) %> add_markers(color = ~cyl)`

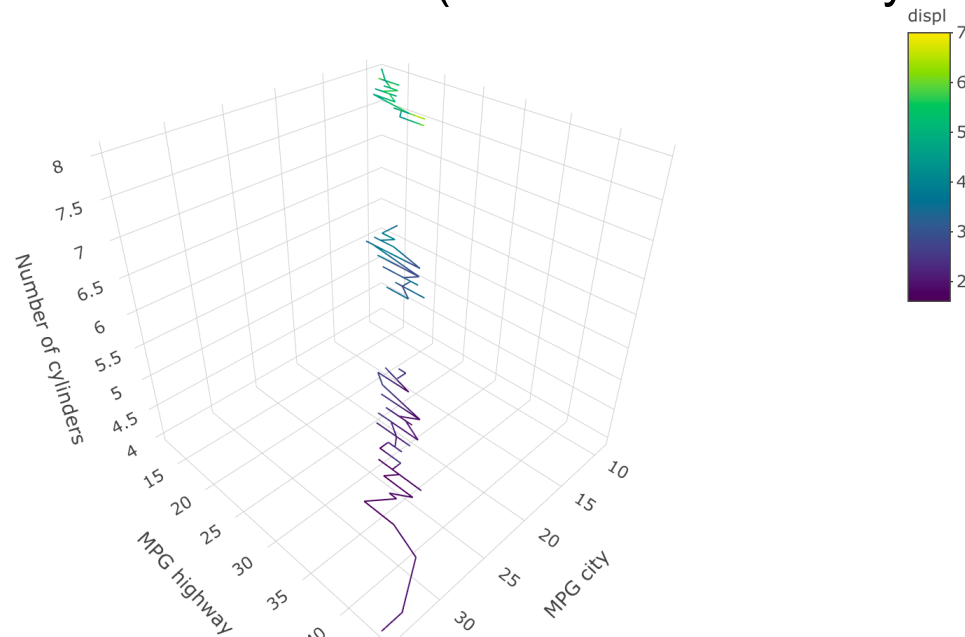


## 2.- Creación de vistas: gráficos 3D

- **Renombrando ejes:**
  - A través de *scene* en *layout()*.

### EJEMPLO:

- ```
plot_ly(mpg, x = ~cty, y = ~hwy, z = ~cyl) %>% group_by(cyl) %>%  
  add_lines(color = ~displ)  
  %>% layout(scene = list( xaxis = list(title = "MPG city"),  
                           yaxis = list(title = "MPG highway"),  
                           zaxis = list(title = "Number of cylinders")))
```

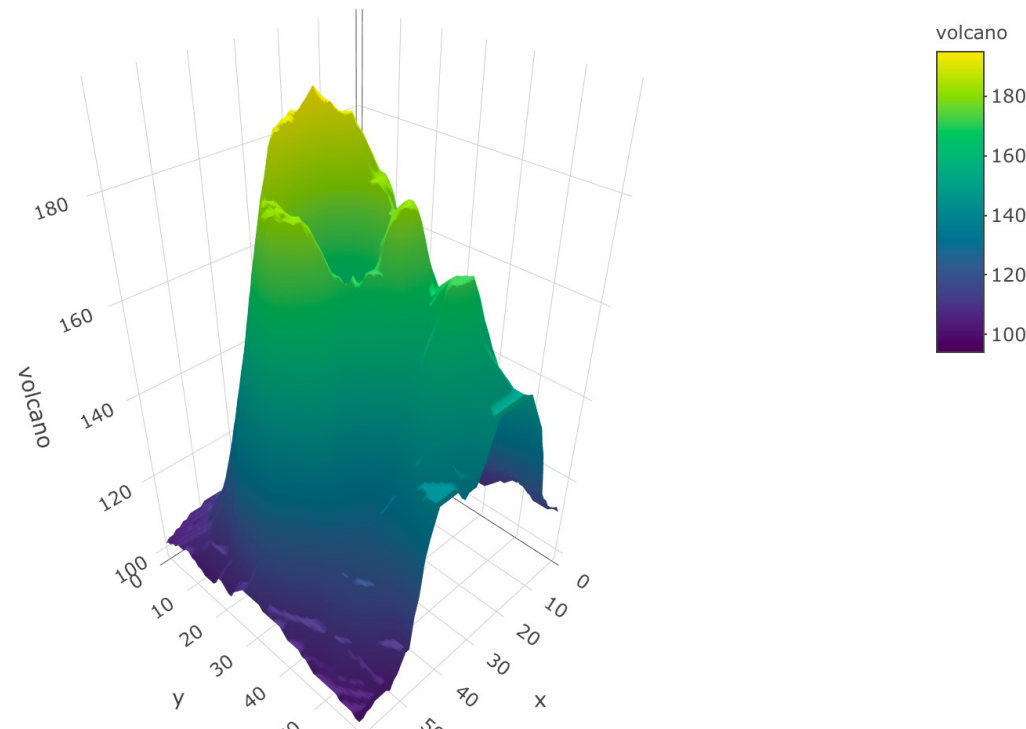


2.- Creación de vistas: gráficos 3D

- **Generación de superficies (`add_surface()`):**
 - Es como crear *heatmaps* en 3D.

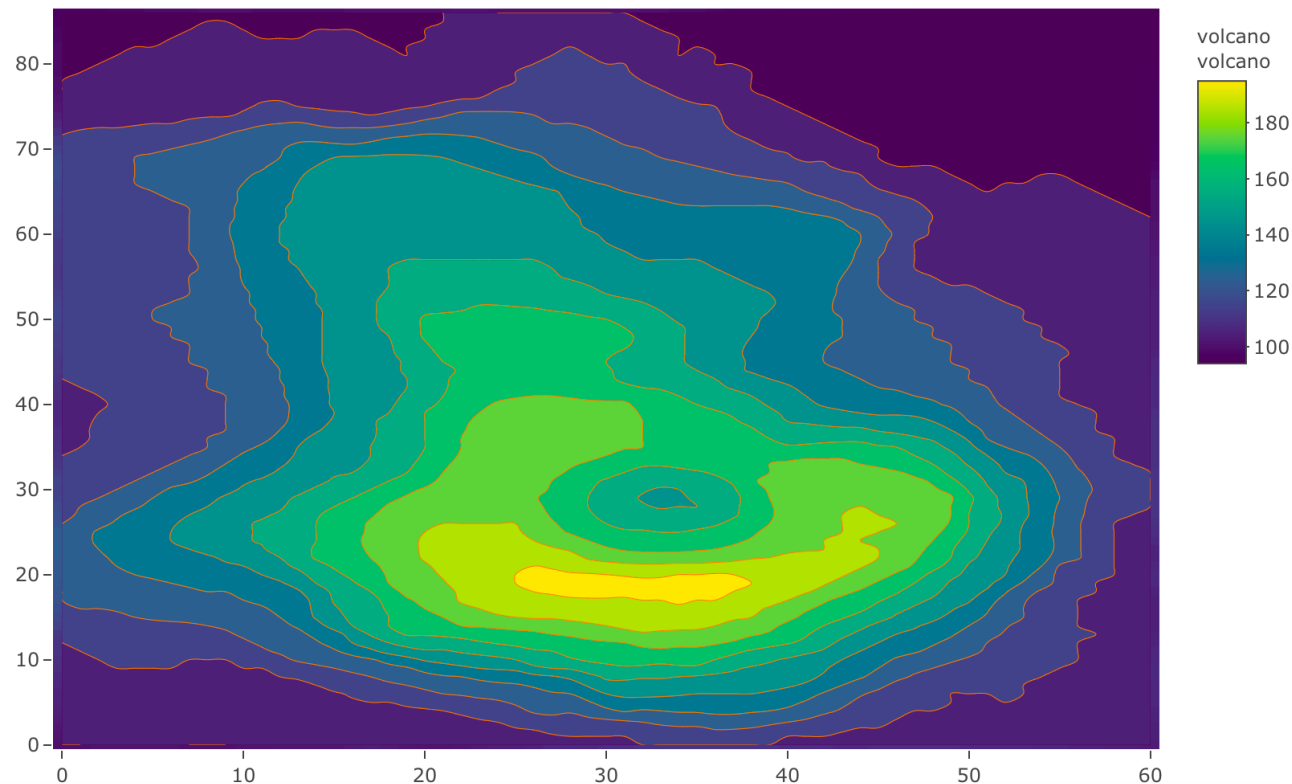
EJEMPLO:

- `x <- seq_len(nrow(volcano))`
- `y <- seq_len(ncol(volcano))`
- `plot_ly() %>% add_surface(x = ~x, y = ~y, z = ~volcano)`



2.- Creación de vistas: gráficos 3D

- **Generación de superficies (*add_surface()*):**
 - Muy espectacular, poco “útil”.
 - Mejor solución:
 - `plot_ly(x=~x, y=~y, z=~volcano) %>% add_heatmap()%>% add_contour(showscale = FALSE)`



PL2 Interactive Graphics.

1 Introducción.

2 Creación de vistas.

3 Publicación de vistas.

4 Combinando múltiples vistas.

5 Conectando múltiples vistas.

3.- Publicación de vistas: dinámicas

- *htmlwidgets::saveWidget()*: permite almacenar generados con **plotly**, **leaflet**, **DT**, ... como fichero HTML autocontenido.
- *partial_bundle()*: permite optimizar la información a almacenar en el fichero HTML a generar.
- *withr::with_dir()*: permite fijar el directorio de almacenamiento.

EJEMPLO:

- ```
withr::with_dir("/Users/paco/temp",
htmlwidgets::saveWidget(partial_bundle(pru), "fichero.html"))
```

## 3.- Publicación de vistas: dinámicas

---

- Si se desea embeber muchos widgets en un documento HTML largo no es recomendable ficheros HTML autocontenidos.
- Mejor crear un único directorio de librerías (dependency files). El navegador sólo tendrá que acceder a ellas una vez.
- Parámetros:
  - *selfcontained* lo pondremos a FALSE.
  - *libdir* lo instanciaremos al directorio de las librerías comunes.
- OJO: incluir el directorio de librerías si se comparte el fichero html con alguien.

## 3.- Publicación de vistas: dinámicas

---

EJEMPLO de HTML no autocontenido:

- `library(htmlwidgets)`
  - `p <- plot_ly(x = rnorm(100))`
  - `saveWidget(p, "p1.html", selfcontained = F, libdir = "lib")`
  - `saveWidget(p, "p2.html", selfcontained = F, libdir = "lib")`
- 
- OJO: incluir el directorio de librerías si se comparte el fichero html con alguien.



## 3.- Publicación de vistas: estáticas

---

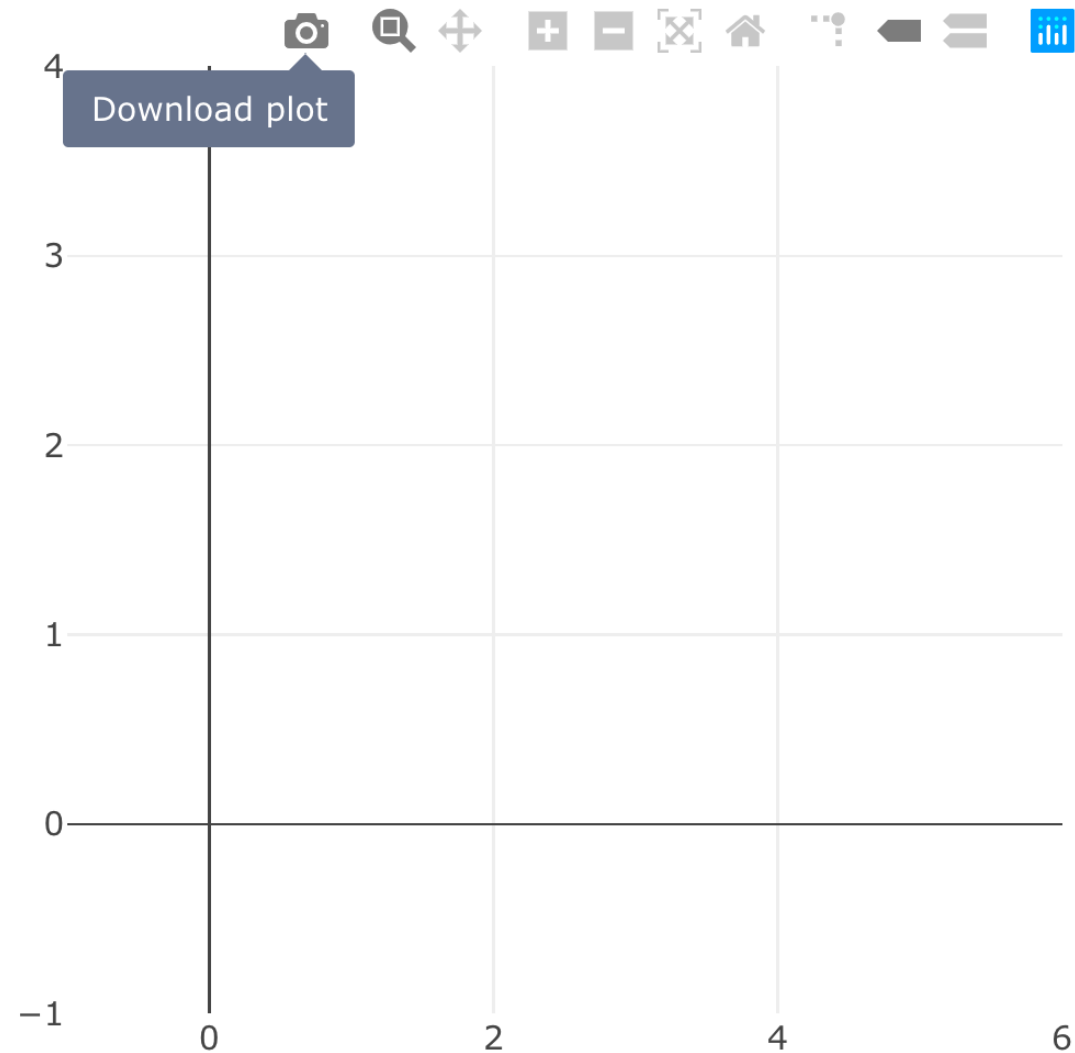
- *config()*: permite fijar las características del fichero a generar.
- Parámetros:
  - *format*: formato del fichero (“svg”, “png”, “jpeg”, “webp”)
  - *filename*: nombre del fichero por defecto.
  - *width*: ancho de la imagen en píxeles.
  - *height*: alto de la imagen en píxeles.

### EJEMPLO:

- `plot_ly() %>%`
- `config( tolImageButtonOptions = list( format = "svg",  
filename = "myplot", width = 600, height = 700))`

## 3.- Publicación de vistas: estáticas

---



# PL2 Interactive Graphics.

---

1 Introducción.

2 Creación de vistas.

3 Publicación de vistas.

4 Combinando múltiples vistas.

5 Conectando múltiples vistas.

## 4.- Combinando vistas: organización

---

- La organización de múltiples vistas en un solo documento permite el **análisis de datos de grandes dimensiones** (reduciendo dimensionalidad).
- Asimismo, el “**linkado**” de estas vistas permite relacionarlas durante el proceso de análisis.
- Pero primero vamos a **aprender a ordenar**.

## 4.- Combinando vistas: organización

---

### ***subplot()***:

- Permite unir múltiples objetos tipo **plotly** en un único objeto.
- Si uno de los objetos a unir es un **ggplot2** lo convierte automáticamente en objeto **plotly**.
- Parámetros (1/2):
  - Los primeros parámetros siempre han de ser objetos **plotly** (incluido otro *subplot()*).
  - *nrows*: indica el número de filas en las que se organizarán los gráficos.
  - *shareX*: indica si se van a compartir (TRUE) o no (FALSE) los ejes X entre las gráficas.
  - *shareY*: indica si se van a compartir (TRUE) o no (FALSE) los ejes Y entre las gráficas.

## 4.- Combinando vistas: organización

---

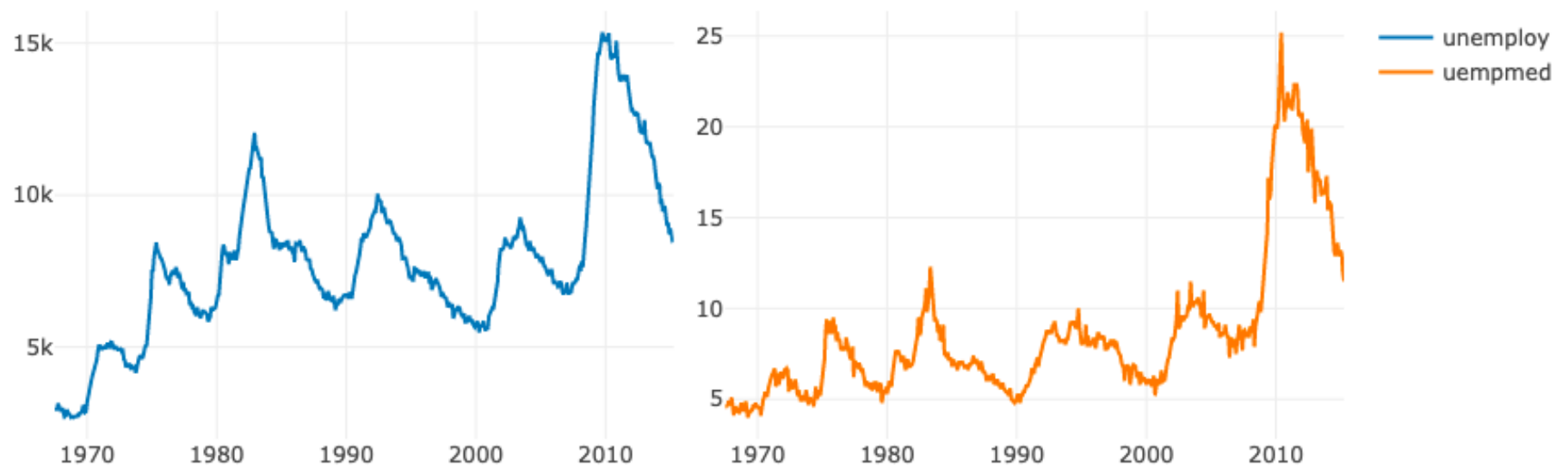
### ***subplot()*:**

- Parámetros (2/2):
  - *titleX*: título de eje X (FALSE si no va a tener).
  - *titleY*: título de eje Y (FALSE si no va a tener).
  - *heights*: distribución en porcentaje de altura de ocupación de cada gráfica (si no se especifica se distribuyen por igual).
  - *widths*: distribución en porcentaje de anchura de ocupación de cada gráfica (si no se especifica se distribuyen por igual).
  - *margin*: margen entre las gráficas.

## 4.- Combinando vistas: organización

### EJEMPLO simple:

- `library(plotly)`
- `p1 <- plot_ly(economics, x = ~date, y = ~unemploy) %>%  
 add_lines(name = "unemploy")`
- `p2 <- plot_ly(economics, x = ~date, y = ~uempmed) %>%  
 add_lines(name = "uempmed")`
- `subplot(p1, p2)`

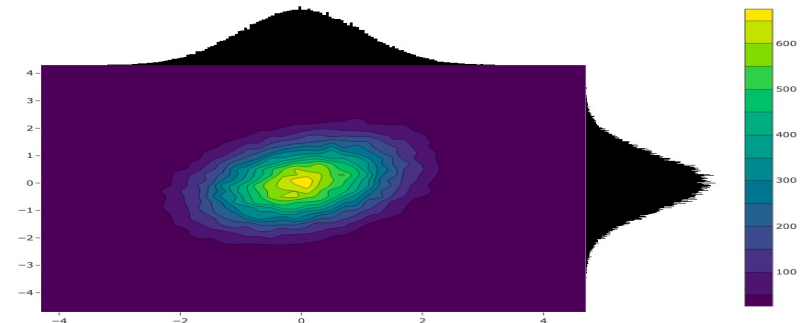


## 4.- Combinando vistas: organización

---

### EJEMPLO complejo:

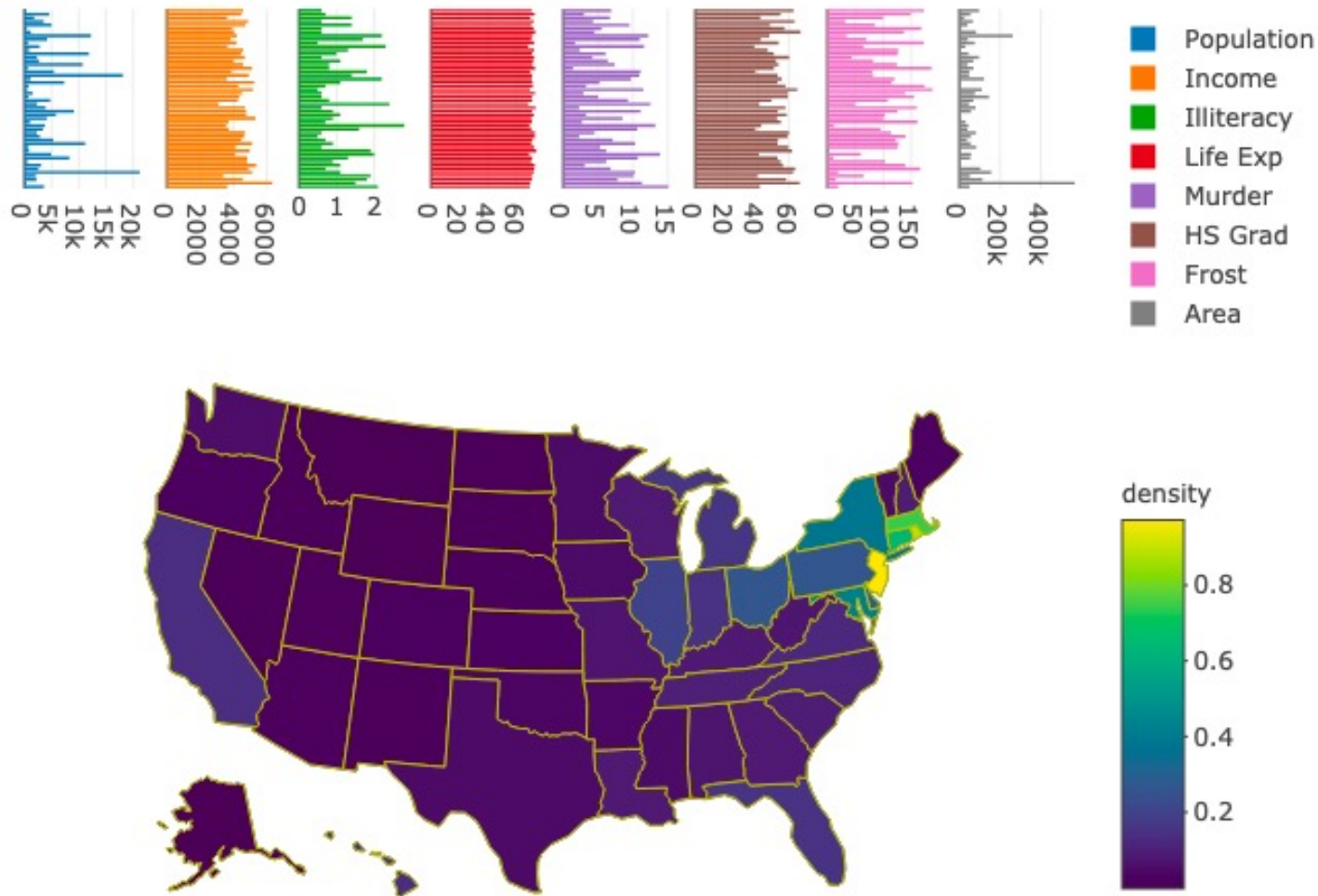
- # draw random values from correlated bi-variate normal distribution
- `s <- matrix(c(1, 0.3, 0.3, 1), nrow = 2)`
- `m <- mvtnorm::rmvnorm(1e5, sigma = s)`
- `x <- m[, 1]`
- `y <- m[, 2]`
- `s <- subplot(plot_ly(x = x, color = I("black")), plotly_empty(),  
plot_ly(x = x, y = y, color = I("black"))) %>%  
add_histogram2dcontour(colorscale = "Viridis"),  
plot_ly(y = y, color = I("black")),  
nrows = 2, heights = c(0.2, 0.8), widths = c(0.8, 0.2), margin = 0,  
shareX = TRUE, shareY = TRUE, titleX = FALSE, titleY = FALSE)`
- `layout(s, showlegend = FALSE)`





## 4.- Combinando vistas: organización

EJEMPLO *subplot()* anidados:

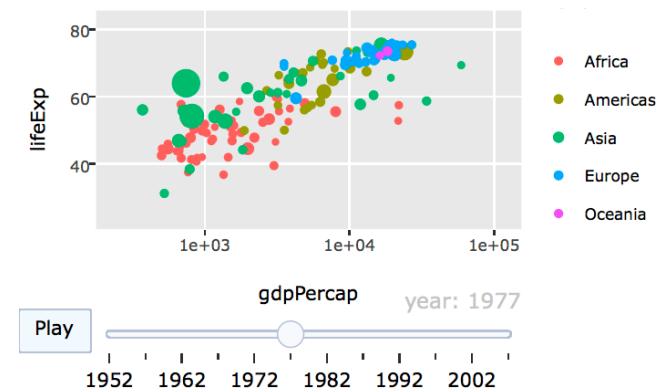


## 4.- Combinando vistas: animación

- `plot_ly()` y `ggplotly()` soportan animación.
- Parámetros:
  - `frame`: argumento/estética al que se asigna un atributo para animación.
  - `ids`: argumento/estética al que se asigna un atributo para conseguir transiciones suaves entre objetos con el mismo `id`.

### EJEMPLO:

- `data(gapminder, package = "gapminder")`
- `gg <- ggplot(gapminder, aes(gdpPercap, lifeExp, color = continent))`  
+ `geom_point(aes(size = pop, frame = year, ids = country))` +  
`scale_x_log10()`
- `ggplotly(gg)`



## 4.- Combinando vistas: animación

---

- Funciones asociadas:
  - *animation\_button()*: configuración del botón de animación (play/pause).
  - *animation\_slider()*: configuración de la barra de deslizamiento.
  - *animation\_opts()*: opciones de animación (tiempos entre “frames”, tiempos de transición, tipo de animación, redibujado, ...).

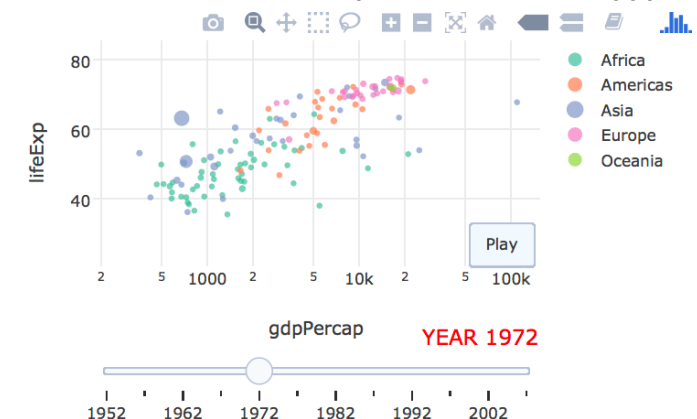
### MÁS INFORMACIÓN:

- [https://www.rdocumentation.org/packages/plotly/versions/4.9.2.1/topics/animation\\_opts](https://www.rdocumentation.org/packages/plotly/versions/4.9.2.1/topics/animation_opts)

## 4.- Combinando vistas: animación

### EJEMPLO:

- `base <- gapminder %>%`
- `plot_ly(x = ~gdpPercap, y = ~lifeExp, size = ~pop,`  
`text = ~country, hoverinfo = "text") %>%`  
`layout(xaxis = list(type = "log"))`
- `base %>%`  
`add_markers(color = ~continent, frame = ~year, ids = ~country) %>%`  
`animation_opts(1000, easing = "elastic", redraw = FALSE) %>%`  
`animation_button( x = 1, xanchor = "right", y = 0, yanchor = "bottom") %>%`  
`animation_slider(currentvalue = list(prefix = "YEAR ", font = list(color="red")))`

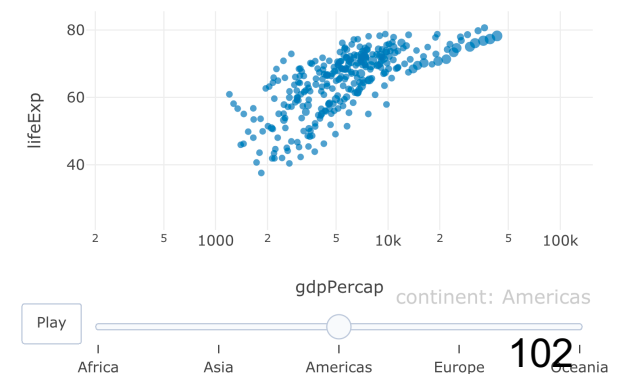


## 4.- Combinando vistas: animación

- Comportamiento parámetro *frame*:
  - Si el valor es numérico ordena en función de éste.
  - Si el valor es alfanumérico ordena alfabéticamente.
  - Si el valor es un factor hay que determinar el orden.

### EJEMPLO:

- `meanLife <- with(gapminder, tapply(lifeExp, INDEX = continent, mean))`
- `gapminder$continent <- factor(  
 gapminder$continent, levels = names(sort(meanLife)))`
- `base %>%  
 add_markers(data = gapminder, frame =  
 ~continent) %>%  
 hide_legend() %>%  
 animation_opts(frame = 1000, transition = 0,  
 redraw = FALSE)`



## 4.- Combinando vistas: animación

---

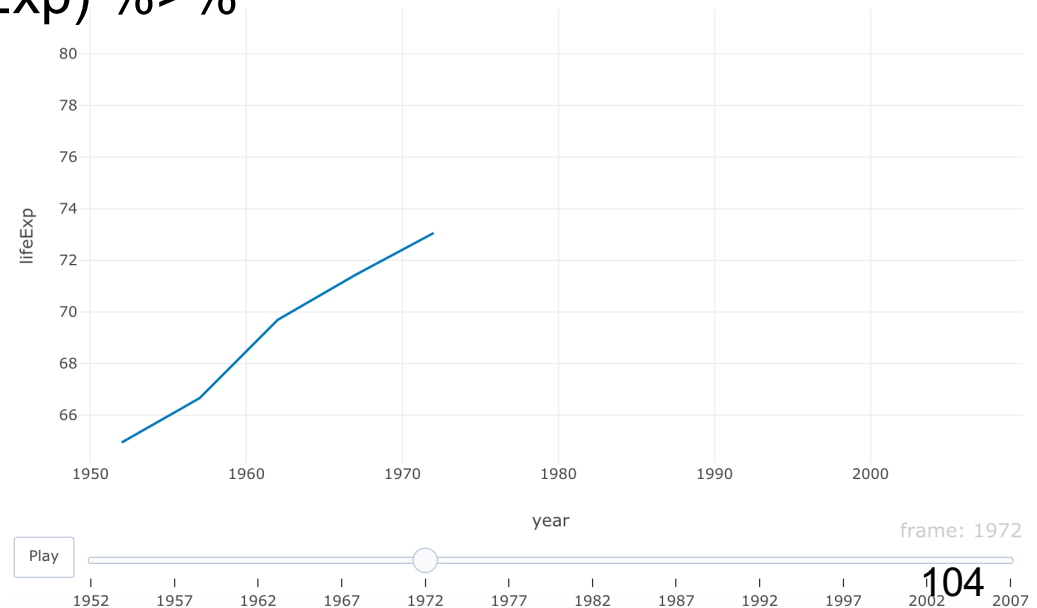
### Animación de gráficos de líneas:

- No hay animación de gráficos de líneas.
- Hay que preparar tantos gráficos como frames.
- Hay que dibujar cada gráfico en secuencia (completo).
- Funciones para preparar la gráfica:
  - *split()*: divide las filas en función del valor de atributo de animación.
  - *bind\_rows()*: permite unir filas de dos dataframes.
  - *accumulate()*: permite acumular filas consecutivas de un dataframe.

## 4.- Combinando vistas: animación

### EJEMPLO:

- `library(dplyr)`
- `library(purrr)`
- `Spain <- filter(gapminder, country == "Spain")`
- `Spain %>% split(.$year) %>% accumulate(., ~bind_rows(.x,.y))`  
    `%>% bind_rows(id = "frame") %>%`  
    `plot_ly(x = ~year, y = ~lifeExp) %>%`  
    `add_lines(frame = ~frame)`



# PL2 Interactive Graphics.

---

- 1 Introducción.
- 2 Creación de vistas.
- 3 Publicación de vistas.
- 4 Combinando múltiples vistas.
- 5 Conectando múltiples vistas.



## 5.- Conectando múltiples vistas.

---

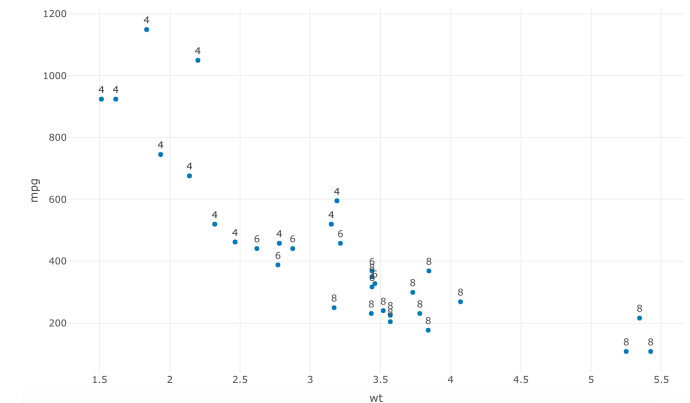
- Es una herramienta muy potente para analizar información de grandes dimensiones.
- Dos aproximaciones relación cliente-servidor:
  - **Servidor**: se transmiten los datos procesados.
  - **Cliente**: se transmiten los datos y se procesan en local.
- **plotly** trabaja desde el punto de vista del cliente.
- Para trabajar en el servidor hay que combinar **plotly** con **shiny**.

- Un *click* lanza la consulta.
- Doble *click* elimina consulta.

- library(plotly)

- ```
– mtcars %>% highlight_key(~cyl) %>%
```

```
plot_ly(x = ~wt, y = ~mpg, text = ~cyl, mode = "markers+text",
        textposition = "top",
        hoverinfo = "x+y") %>%
highlight(on = "plotly_hover",
         off = "plotly_doubleclick")
```



5.- Conectando múltiples vistas.

- Cualquier **interacción** (fijada por el parámetro *on* de la función *highlight()*) que se lleva a cabo sobre el gráfico se traduce en una activación (**destacado**) de todos los elementos del gráfico que responden a la **consulta**:

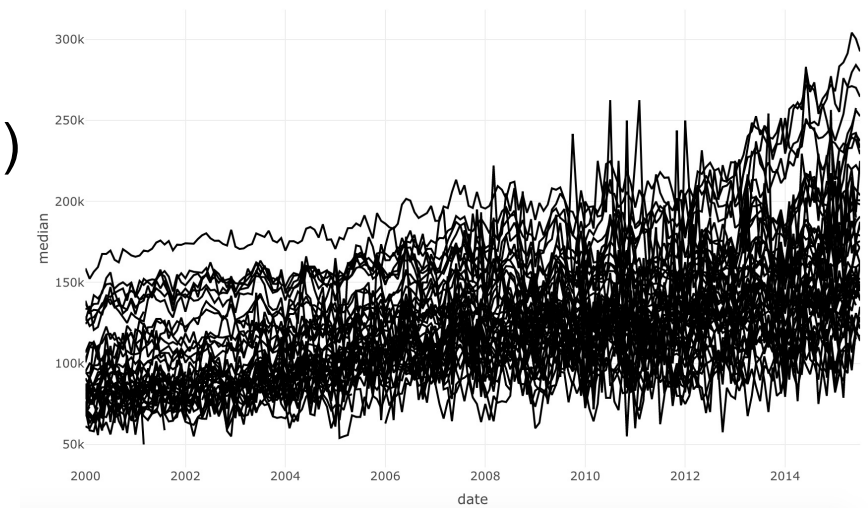
SELECT * FROM *dataframe* **WHERE**
elemento_highlight_key **IN** *selección_interacción*

5.- Conectando múltiples vistas.

SELECT * FROM *dataframe* WHERE
elemento_highlight_key **IN** *selección_interacción*

EJEMPLO:

- `data(txhousing, package = "ggplot2")`
- `tx <- highlight_key(txhousing, ~city)`
- `base <- plot_ly(tx, color = I("black")) %>% group_by(city)`
- `time_series <- base %>%
 add_lines(x = ~date, y = ~median)`



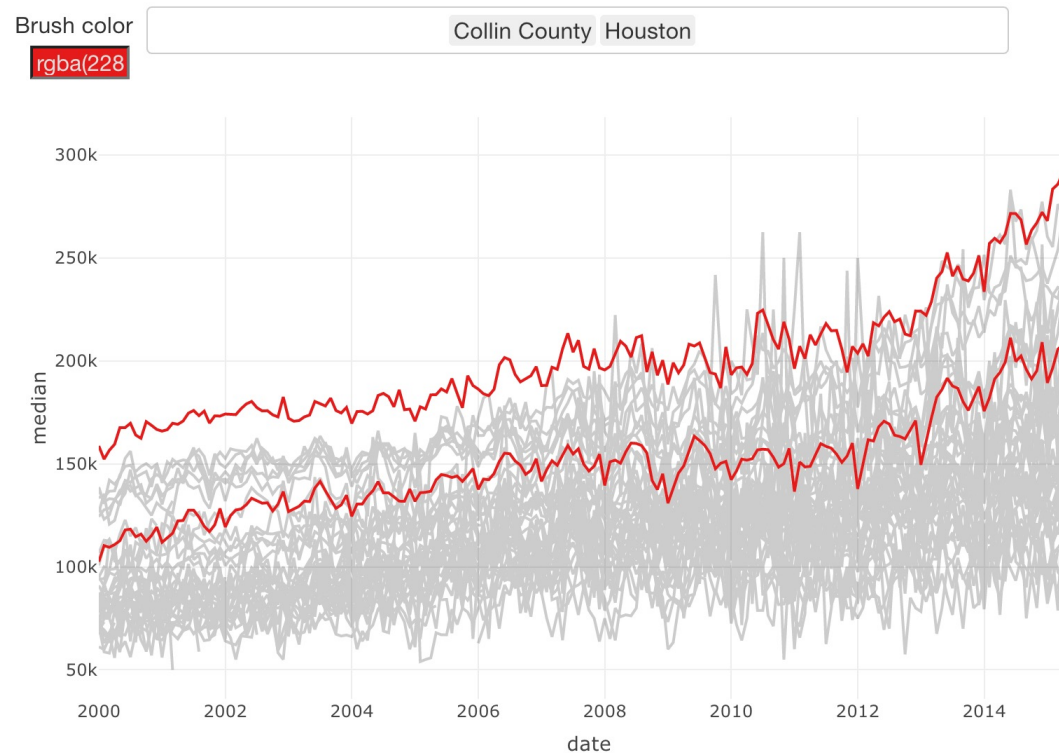
5.- Conectando múltiples vistas.

- Parámetros de la función *highlight()*:
 - El primer parámetros siempre es el objeto **plotly**.
 - *on*: acción que lanza la consulta.
 - *off*: acción que anula la consulta.
 - *selectize*: crea una lista desplegable con los posibles valores de la consulta.
 - *dynamic*: (true o false) asigna un color dinámicamente a los elementos destacados.
 - *persistent*: (true o false) indica si la selección se hace persistente o se anula cuando se hace otra selección.

5.- Conectando múltiples vistas.

EJEMPLO:

- `highlight(serie, on = "plotly_click", selectize = TRUE, dynamic = TRUE, persistent = TRUE)`



5.- Conectando múltiples vistas.

EJEMPLO 1 (linkando 2 vistas):

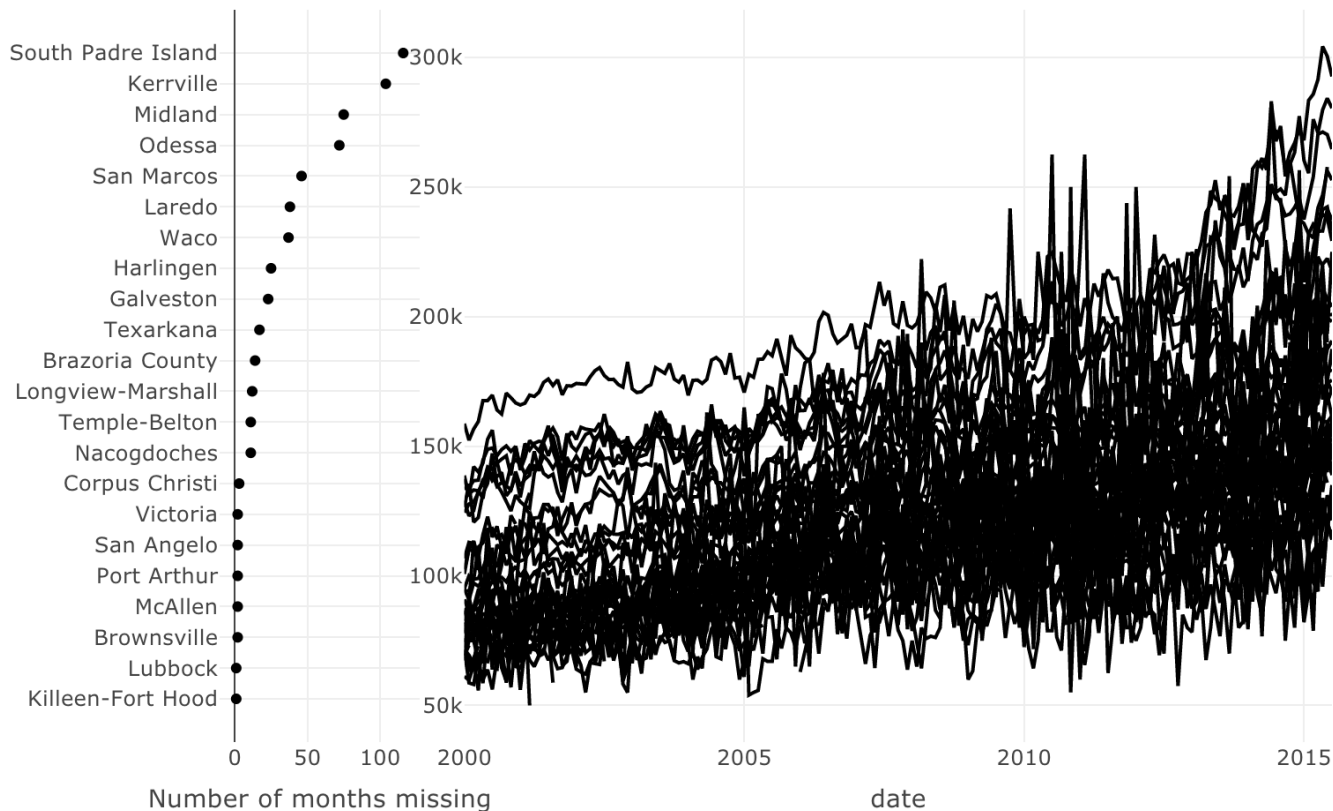
- ```
dot_plot <- base %>% summarise(miss = sum(is.na(median)))
%>% filter(miss > 0) %>%
 add_markers(x = ~miss, y = ~forcats::fct_reorder(city, miss),
 hoverinfo = "x+y") %>%
 layout(xaxis = list(title = "Number of months missing"),
 yaxis = list(title = ""))
```
- ```
subplot(dot_plot, time_series, widths = c(.2, .8), titleX = TRUE)  
%>% layout(showlegend = FALSE) %>%  
  highlight(on = "plotly_selected", dynamic = TRUE, selectize =  
            TRUE)
```

5.- Conectando múltiples vistas.

EJEMPLO 1 (linkando 2 vistas):

Brush color

rgba(228

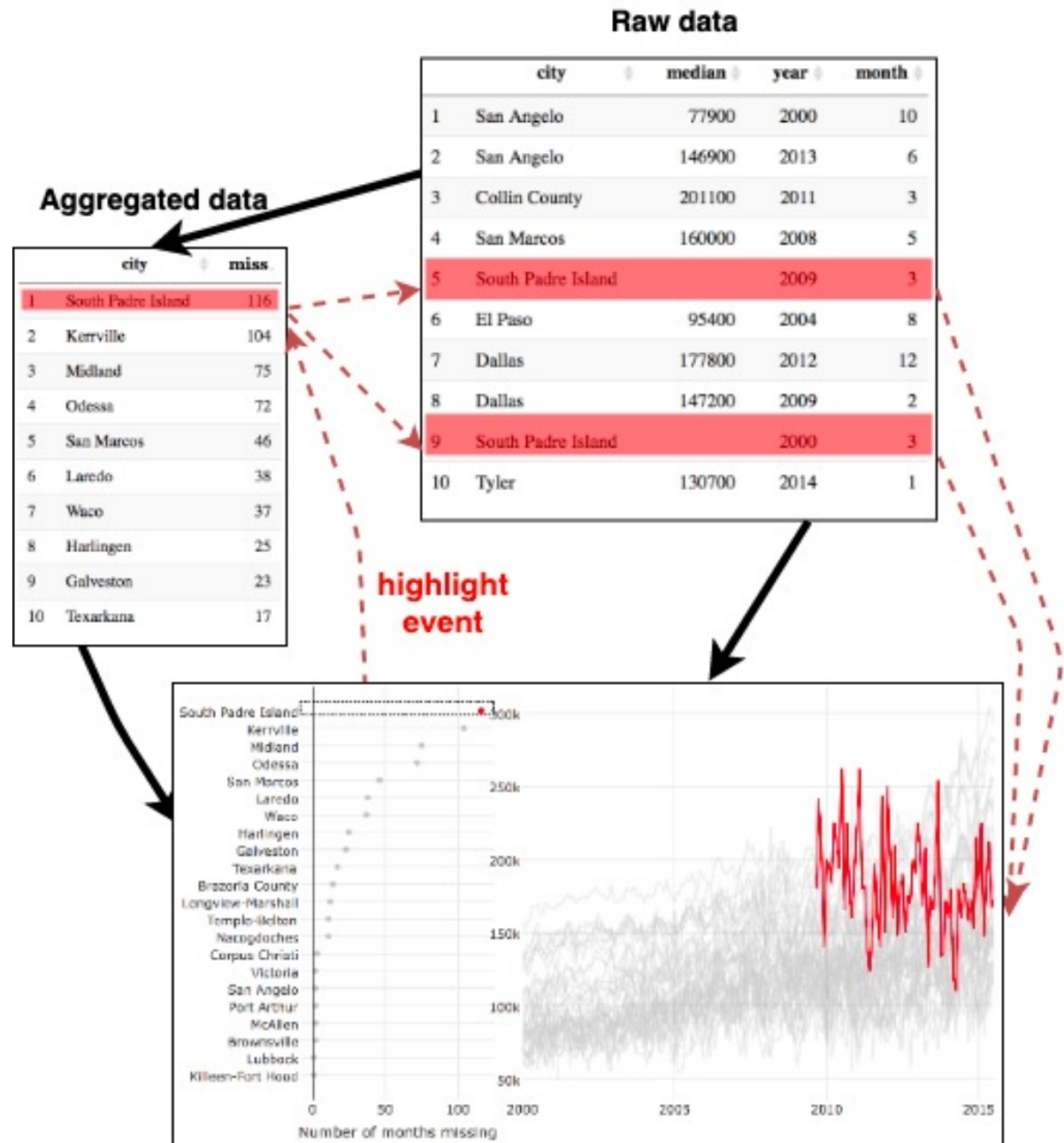


5.- Conectando múltiples vistas.

EJEMPLO 1

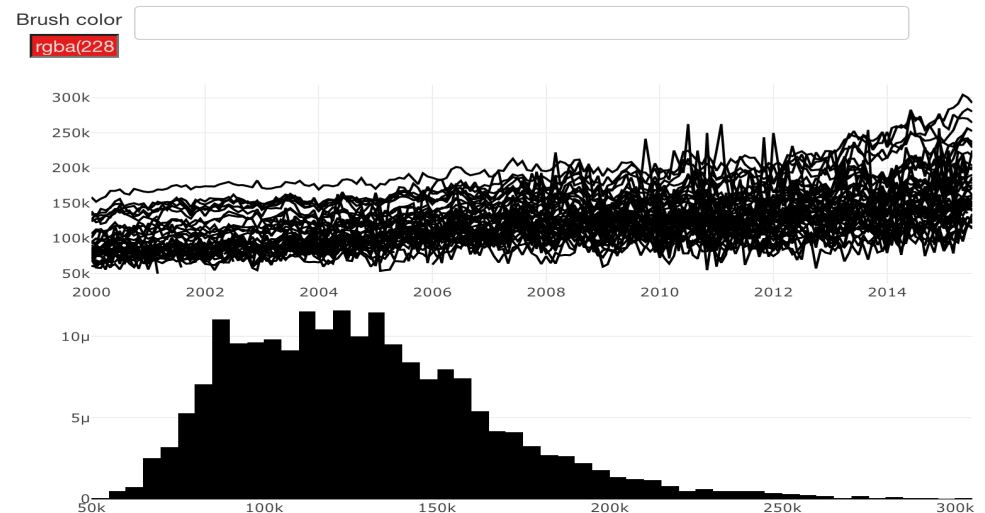
(linkando 2 vistas):

- Cómo funciona la consulta.



EJEMPLO 2 (linkando 2 vistas):

- `hist <- add_histogram( base, x = ~median, histnorm = "probability density")`
- `subplot(time_series, hist, nrow = 2) %>% layout(barmode = "overlay", showlegend = FALSE) %>% highlight(dynamic = TRUE, selectize = TRUE, selected = attrs_selected(opacity = 0.3))`



5.- Conectando múltiples vistas.

Highlight vs Eventos de Filtrado:

- **Highlight** atenúa la opacidad de las marcas existentes, y luego añade una capa gráfica adicional que representa la selección.
- Un evento de filtro elimina completamente las marcas existentes y reajusta los ejes a los datos restantes.
- Para el reajuste usando *ggplotly()* se necesita especificar *dynamicTicks = TRUE*.
- Un evento de filtrado debe ser ejecutado por un widget de filtro de la librería **crosstalk**.

5.- Conectando múltiples vistas.

Highlight vs Eventos de Filtrado:

EJEMPLO:

- `library(crosstalk)`
- `tx <- highlight_key(txhousing)`
- `gg <- ggplot(tx) + geom_line(aes(date, median, group = city))`
- `filter <- bscols(filter_select("id", "Select a city", tx, ~city),
ggplotly(gg, dynamicTicks = TRUE),
widths = c(12, 12))`
- `tx2 <- highlight_key(txhousing, ~city, "Select a city")`
- `gg <- ggplot(tx2) + geom_line(aes(date, median, group = city))`
- `select <- highlight(ggplotly(gg, tooltip = "city"), selectize = TRUE,
persistent = TRUE)`
- `bscols(filter, select)`

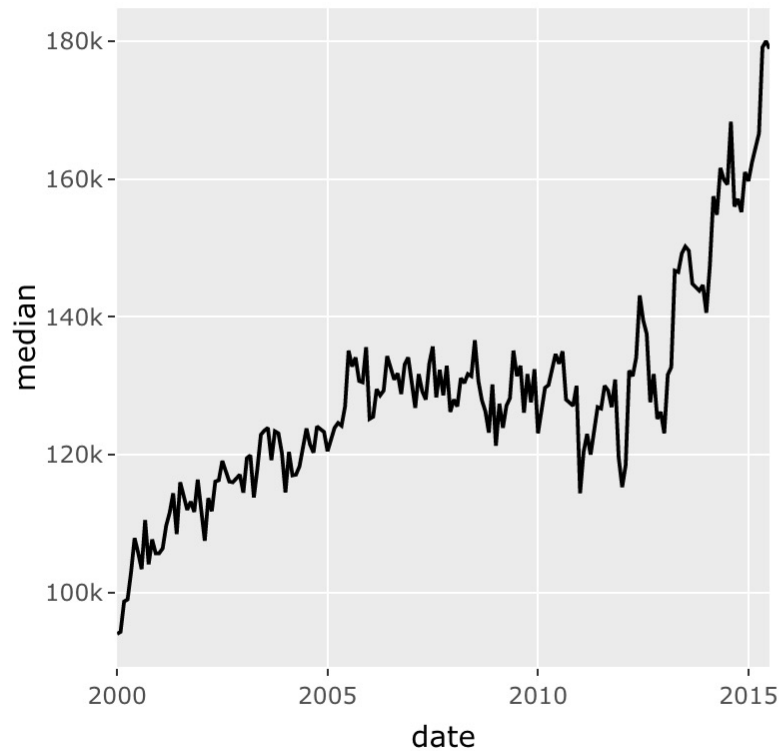
5.- Conectando múltiples vistas.

Highlight vs Eventos de Filtrado:

EJEMPLO:

Select a city

Arlington



Select a city

Arlington



5.- Conectando múltiples vistas.

Eventos de Filtrado:

- *filter_select()*: permite seleccionar valores mediante una ventana desplegable (nominales o factores).
- *filter_slider()*: permite seleccionar un rango de valores mediante una barra de deslizamiento (numéricos).
- *filter_checkbox()*: permite seleccionar valores mediante selección de casillas.
- Parámetros:
 - El primer parámetro es el identificador html.
 - El segundo parámetro es el texto de la casilla de filtrado.
 - El tercer parámetro es el dataframe.
 - El cuarto parámetro es el atributo del dataframe a seleccionar.

5.- Conectando múltiples vistas.

Eventos de Filtrado:

EJEMPLO:

- `library(crosstalk)`
- `tx <- highlight_key(txhousing)`
- `widgets <- bscols(widths = c(12, 12, 12),
 filter_select("city", "Cities", tx, ~city),
 filter_slider("sales", "Sales", tx, ~sales),
 filter_checkbox("year", "Years", tx, ~year, inline = TRUE))`
- `bscols(
 widths = c(4, 8), widgets,
 plot_ly(tx, x = ~date, y = ~median, showlegend = FALSE) %>%
 add_lines(color = ~city, colors = "black"))`

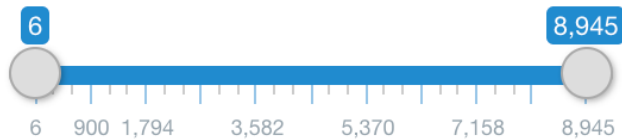
5.- Conectando múltiples vistas.

Eventos de Filtrado:

EJEMPLO:

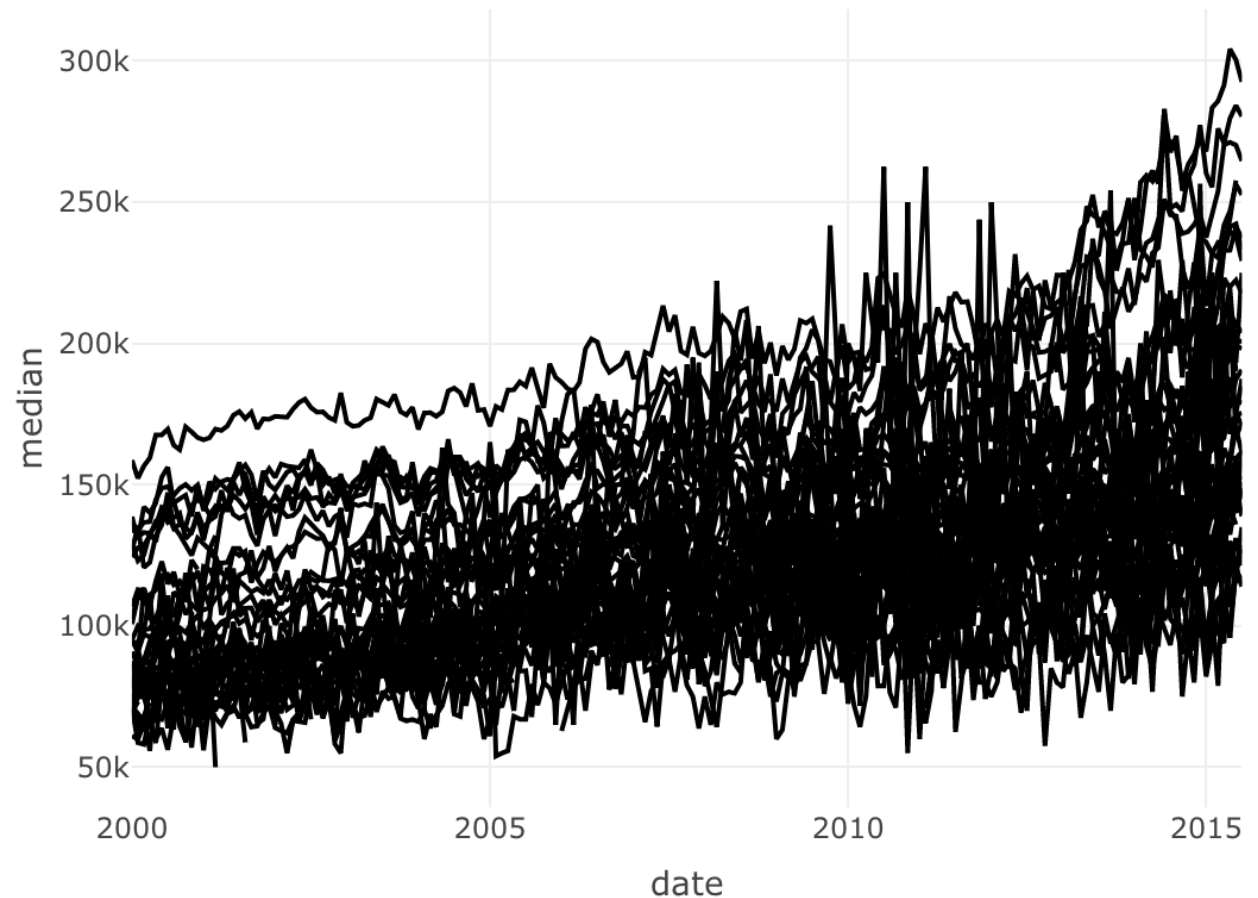
Cities

Sales



Years

- ☐ 2000 ☐ 2001 ☐ 2002 ☐ 2003
☐ 2004 ☐ 2005 ☐ 2006
☐ 2007 ☐ 2008 ☐ 2009
☐ 2010 ☐ 2011 ☐ 2012
☐ 2013 ☐ 2014 ☐ 2015



5.- Conectando múltiples vistas.

Highlight y Eventos de Filtrado pueden trabajar conjuntamente.

EJEMPLO:

```
library(leaflet)

eqs <- highlight_key(quakes)
stations <- filter_slider("station", "Number of Stations", eqs, ~stations)

p <- plot_ly(eqs, x = ~depth, y = ~mag) %>%
  add_markers(alpha = 0.5) %>%
  highlight("plotly_selected")

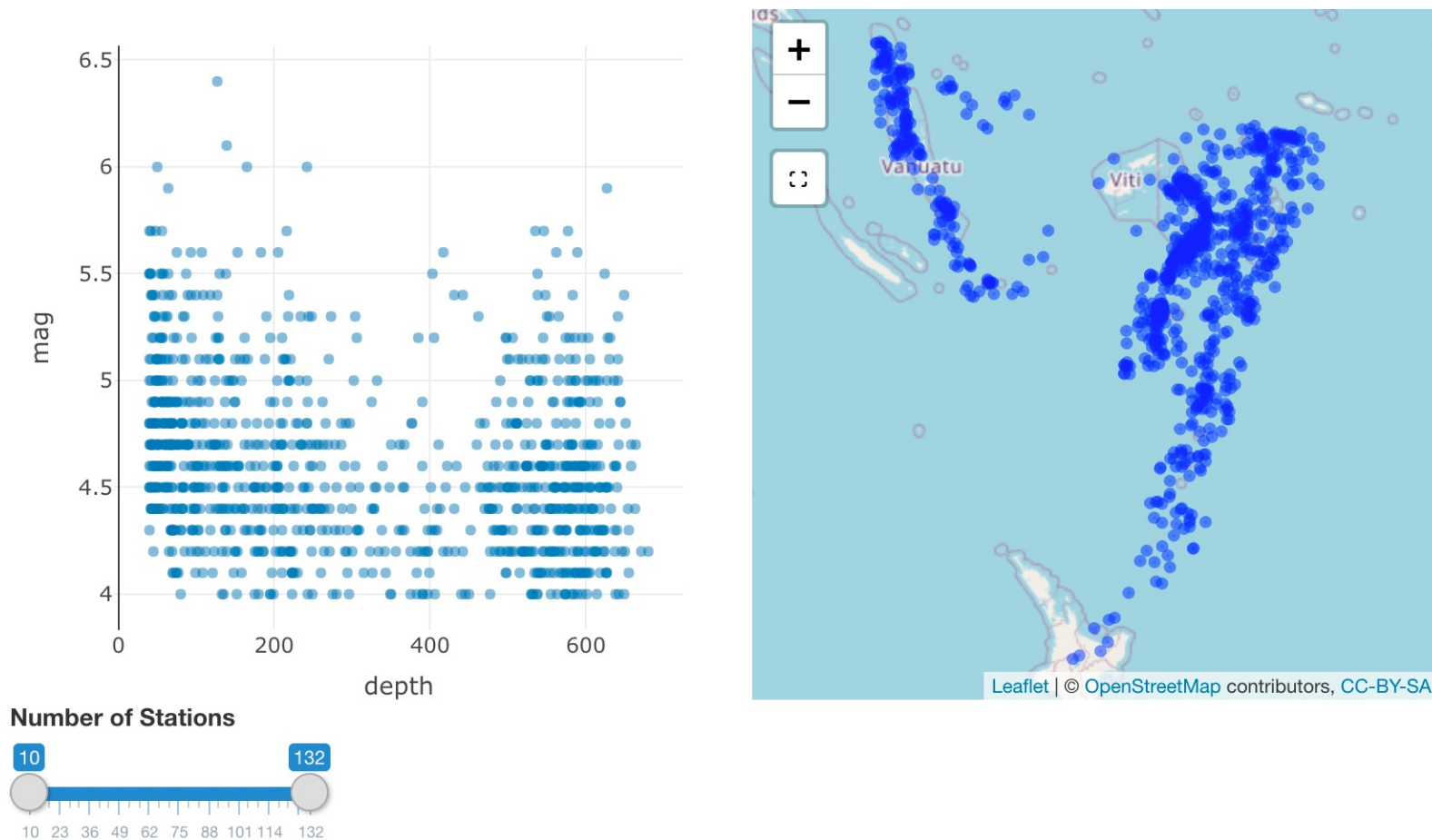
map <- leaflet(eqs) %>% addTiles() %>% addCircles()

bscols(widths = c(6, 6, 3), p, map, stations)
```

5.- Conectando múltiples vistas.

Highlight y Eventos de Filtrado pueden trabajar conjuntamente.

EJEMPLO:



5.- Conectando múltiples vistas.

Highlight y Eventos de Filtrado pueden trabajar conjuntamente.

- **Condición:** la variable *highlight()* debe ser subconjunto de la de filtrado.

EJEMPLO:

```
library(gapminder)
g <- highlight_key(gapminder, ~country)
continent_filter <- filter_select("filter", "Select a country", g, ~continent)

p <- plot_ly(g) %>% group_by(country) %>%
  add_lines(x = ~year, y = ~lifeExp, color = ~continent) %>%
  layout(xaxis = list(title = "")) %>%
  highlight(selected = attrs_selected(showlegend = FALSE))

bscols(continent_filter, p, widths = 12)
```

5.- Conectando múltiples vistas.

Highlight y Eventos de Filtrado pueden trabajar conjuntamente.

- **Condición:** la variable *highlight()* debe ser subconjunto de la de filtrado.

EJEMPLO:

Select a country

